Mendelova univerzita v Brně

Provozně ekonomická fakulta

Tvorba datasetu pre tréning modelov umelej inteligencie v agronómii

Bakalárska práca

Vedúci práce:

doc. Ing. Oldřich Trenz, Ph.D.

Filip Kosar

Brno 2025

ČESTNÉ PREHLÁSENIE

Odovzdaním potvrdzujem, že som prácu **Tvorba datasetu pre tréning modelov umelej inteligencie v agronómii** vypracoval samostatne a všetky použité zdroje a informácie uvádzam v zozname použitej literatúry. Súhlasím, aby moja práca bola zverejnená v súlade s § 47b zákona č. 111/1998 Sb., o vysokých školách, a o zmene a doplnení niektorých zákonov (zákon o vysokých školách), v znení neskorších predpisov, a v súlade s platnou Směrnicí o zveřejňování závěrečných prací.

Som si vedomý, že sa na moju prácu vzťahuje zákon č. 121/2000 Sb., autorský zákon, a že Mendelova univerzita v Brně má právo na uzatvorenie licenčnej zmluvy a použitie tejto práce ako školského diela podľa § 60 odst. 1 autorského zákona.

Ďalej sa zaväzujem, že pred spísaním licenčnej zmluvy o použití diela inou osobou (subjektom) si vyžiadam písomné stanovisko univerzity, že predmetná licenčná zmluva nie je v rozpore s oprávnenými záujmami univerzity, a zaväzujem sa uhradiť prípadný príspevok na úhradu nákladov spojených so vznikom diela, a to až do ich skutočnej výšky.

V Brne dňa 29. decembra 2025 .....

podpis

Poďakovanie

Chcel by som vyjadriť úprimnú vďaku vedúcemu mojej bakalárskej práce, pánovi doc. Ing. Oldřichovi Trenzovi, Ph.D. za jeho pomoc pri vypracovávaní tejto práce, rady a pripomienky, a v neposlednej rade za venovaný čas v podobe odborných konzultácií každý týždeň. Taktiež chcem poďakovať pánovi Ing. Jakubovi Beránkovi, Ph. D. z Ústředního kontrolního a zkušebního ústavu zemědělského, ktorý mi pomohol s orientáciou na ich stránkach a celkovým uchopením mojej témy, vrátane s pánom Ing. Miroslavom Jarošom, ktorý stojí za usmernením v rámci štúdia umelej inteligencie a strojového učenia. Chcem v neposlednej rade veľmi poďakovať aj mojej priateľke, rodine a kamarátom, ktorí ma neuveriteľne podporovali pri tvorbe tejto práce.

ABSTRAKT

Kosar F. *„Tvorba Datasetu pre Tréning Modelov Umelej Inteligencie v Agronómii“*. Bakalárska práca. Brno: Mendelova univerzita, 2025.

Táto bakalárska práca sa zaoberá problematikou tvorby a komplexného spracovania dát do formy multimodálneho datasetu určeného pre trénovanie model umelej inteligencie v oblasti agronómie. Cieľom práce je integrácia zdrojov dát o výskyte hraboša poľného a plesne zemiakovej na území Českej republiky v rokoch 2023 a 2024 so satelitnými snímkami Sentinel-2, vypočítanými vegetačnými indexami a meteorologickými údajmi cez rozhranie Open-Meteo API o podnebí počas konkrétnych záznamov. Práca popisuje proces predspracovania dát, ktoré zahŕňa čistenie, filtrovanie oblačnosti, anotáciu datasetu, proces transformácie na binárny klasifikátor a stratifikované rozdelenie na podmnožiny za účelom trénovania. Tréning je optimalizovaný pre maximalizáciu efektivity tréningu a následne vyhodnotený pomocou klasifikačných metrík. Obsahom je finálne rozhodnutie o potenciálnom využití a možných zlepšeniach.

Kľúčové slová: Umelá inteligencia, Neurónové siete, Hlboké učenie, Predikcia výskytu škodcov, Hraboš poľný, Pleseň zemiaková, Diaľkový prieskum

ABSTRACT

Kosar F. *„Creation of a Dataset for Training Artificial Intelligence Models in Agronomy“.* Bachelor's Thesis. Brno: Mendel University, 2025.

This bachelor's thesis addresses the problem of creating and comprehensively processing data into a multimodal dataset designed for training artificial intelligence models in the field of agronomy. The objective of the work is to integrate data sources regarding the occurrence of the common vole and potato late blight in the Czech Republic during the years 2023 and 2024 with Sentinel-2 satellite imagery, calculated vegetation indices, and meteorological data via the Open-Meteo API providing climate information for specific records. The thesis describes the data preprocessing process, which includes cleaning, cloud filtering, dataset annotation, the process of transformation into a binary classifier, and stratified splitting into subsets for training purposes. The training is optimized to maximize training efficiency and is subsequently evaluated using classification metrics. The content includes a final decision on potential utilization and possible improvements.

Keywords: Artificial Intelligence, Neural Networks, Deep learning, Pest occurence prediction, Field vole, Potato blight, Remote sensing

Obsah

1 Úvod a cieľ práce ... 8

1.1 Úvod ... 8

1.2 Cieľ ... 8

2 Literárna rešerš ... 9

2.1 Význam ochrany rastlín ... 9

2.2 Škodci v agronomickom priemysle ... 9

2.2.1 Hraboš poľný ... 9

2.2.2 Pleseň zemiaková ... 9

2.3 Diaľkový prieskum Zeme v agronómii ... 10

2.3.1 Zber satelitných dát ... 10

2.3.2 Vegetačné indexy ... 10

2.3.3 Meteorologické údaje ... 11

2.4 Strojové učenie ... 11

2.5 Neurónové siete a hlboké učenie ... 12

2.5.1 Neurón a neurónová sieť ... 12

2.5.2 Proces učenia ... 14

2.5.3 Deep learning ... 15

2.6 Architektúry hlbokého učenia ... 15

2.6.1 Konvolučné neurónové siete ... 15

2.6.2 Transforméry ... 16

2.6.3 Transfer learning ... 16

2.7 Optimalizácia a hodnotenie modelov ... 17

2.7.1 Overfitting a underfitting ... 17

2.7.2 Metriky hodnotenia ... 18

2.8 Úloha datasetu v ML ... 19

2.8.1 Štruktúra datasetov v agronómii ... 19

2.8.2 Anotácia dát ... 20

2.9 Predspracovanie a čistenie ... 20

2.9.1 Integrácia ... 20

2.9.2 Normalizácia a štandardizácia ... 21

2.9.3 Problém neúplných a nevyhovujúcich dát ... 21

2.10 Rozdelenie datasetu ... 21

2.10.1 Trénovacia množina ... 22

2.10.2 Validačná množina ... 22

2.10.3 Testovacia množina ... 22

2.11 Nevyváženosť tried ... 22

3 Metodika práce ... 23
3.1 Analýza oblasti ... 23
3.2 Spracovanie vstupných dát ... 23
3.3 Návrh a implementácia štruktúry datasetu ... 24
3.4 Overenie kvality datasetu ... 24
4 Vlastné riešenie ... 25
4.1 Použité nástroje a knižnice ... 25
4.2 Prvý zdroj dát – terénne hlásenia ... 26
4.2.1 ÚKZÚZ ... 26
4.2.2 Integrácia terénnych hlásení ... 27
4.2.3 Zmena formátu ... 27
4.3 Druhý zdroj dát – satelitné snímky a vegetačné indexy ... 28
4.3.1 Sentinel-2 ... 28
4.3.2 Integrácia pomocou GEE ... 28
4.3.3 Prečistenie vysokej oblačnosti ... 31
4.4 Tretí zdroj dát – meteorologické údaje ... 33
4.4.1 Prepojenie cez Open-Meteo API ... 33
4.4.2 Problém chýbajúcich dát ... 36
4.5 Riešenie nízkej kvality satelitných snímok ... 37
4.5.1 Nový zdroj – CDSE ... 37
4.6 Spracovanie pre finálnu formu datasetu ... 41
4.6.1 Výskyt pretransformovaný na binárny klasifikátor ... 41
4.6.2 Konflikt množstva záznamov škodcov ... 42
4.6.3 Pripojenie merateľnej intenzity výskytu ... 42
4.6.4 Kontrola integrity ... 42
4.6.5 0/1 anotácia ... 43
4.6.6 Filtrovanie oblačnosti snímok CDSE ... 45
4.6.7 Rozdelenie datasetu na Train/Val/Test ... 46
4.7 Učenie ... 47
4.7.1 ResNet ... 47
4.7.2 MLP ... 47
4.7.3 Proces učenia ... 47
4.8 Vyhodnotenie ... 53
5 Diskusia ... 56
5.1 Identifikované problémy ... 56
5.2 Možnosti zlepšenia ... 56
6 Záver ... 57
7 Zoznamy a literatúra ... 58

1 Úvod a cieľ práce

1.1 Úvod

Poľnohospodárstvo je v posledných rokoch čoraz viac vystavované tlaku klimatických zmien, ktoré ovplyvňujú priebeh vegetácie a šírením škodlivých organizmov (Skendžić et al., 2021). Agronómia preto stále viac využíva technológie, ktoré umožňujú efektívnejšie monitorovanie a predikciu výskytu škodcov. Jedným z najvýznamnejších a najaktuálnejších spôsobov identifikácie problematických poľnohospodárskych plôch je využívanie umelej inteligencie. Za pomoci metód strojového učenia je možné určovať rýchlejšie a súčasne s vyššou istotou poškodenie a stres plodín alebo výskyt škodlivých organizmov. Kvalita modelu ale priamo závisí od vstupných dát, na ktorých prebieha tréning.

Tvorba vhodných datasetov pre agronómiu je proces, ktorý si vyžaduje prepojenie rôznych zdrojov informácií – od pozorovaní výskytu škodcu cez meteorologické podmienky až po stav vegetácie daných oblastí. Moderné prístupy preto zdôrazňujú integráciu rozličných dátových zdrojov, pričom dátová fúzia sa ukazuje ako dôležitý nástrojov pre zlepšovanie monitorovania, klasifikácie plodín alebo predikcie rušivých elementov (Allu et al., 2024).

Táto bakalárska práca sa zameriava na praktickú stránku tvorby datasetu pre trénovanie modelov umelej inteligencie v agronómii. V rámci riešenia boli zhromaždené a spracované záznamy o výskyte hraboša poľného a plesne zemiakovej v Českej republike. Dôraz bol kladený na integráciu a predspracovanie dát, anotáciu datasetu a následný tréning pre vyhodnotenie vytvoriť robustný základ pre následné experimenty s modelmi strojového učenia.

1.2 Cieľ

Cieľom tejto bakalárskej práce je navrhnúť a realizovať proces tvorby datasetu zameraného na agronomické aplikácie, konkrétne na výskyt vybraných škodlivých organizmov, pričom tento dataset bude prepojený so satelitnými a environmentálnymi dátami a následne využitý na trénovanie a overenie základného modelu umelej inteligencie v agronómii.

2 Literárna rešerš

2.1 Význam ochrany rastlín

Ochrana rastlín je dôležitým prvkom potravinovej bezpečnosti a významná časť svetovej produkcie je každoročne ohrozovaná rôznymi negatívnymi faktormi. Podľa analýzy Oerkeho (2006) môžu straty na úrode spôsobené škodcami dosiahnuť 40-50%, avšak plošná aplikácia chemikálií prináša ekologické riziká a záťaž v ekonomike. Riešením je presné poľnohospodárstvo, ktoré využíva dáta z prieskumu Zeme a algoritmy strojového učenia na špecifickú aplikáciu ochranných látok, čím sa zefektívňuje počet zásahov spolu s minimalizáciou vstupov (Mulla, 2012).

2.2 Škodci v agronomickom priemysle

Ochrana rastlín si vyžaduje včasnú identifikáciu škodcov, ktorí predstavujú riziko pre poľnohospodársku produkciu. Monitorovanie týchto škodlivých organizmov je zväčša na základe manuálnych prehliadok polí a sledovania prahov škodlivosti, ale modernými technológiami sa začínajú viac využívať spôsoby precízneho poľnohospodárstva využívajúce umelú inteligenciu na plošnú a efektívnejšiu kontrolu (Xu & Wang, 2025). Na uvedomenie si potreby detekcie pomocou umelej inteligencie je dôležité si popísať škodlivé organizmy a ich spôsoby šírenia.

2.2.1 Hraboš poľný

Tento cicavec je v strednej Európe považovaný za jedného z najvýznamnejších poľnohospodárskych škodcov. Škody spôsobuje hlavne konzumovaním koreňov a nadzemných častí plodín, čo spôsobuje menšiu alebo žiadnu neúrodnosť. Populácia hrabošov sa vyznačuje kruhovitými útvarmi ohlodaných plodín poľa a podľa ÚKZÚZ (2023) za príčinou premnoženia stoja zmeny klímy, kedy mierne zimy uľahčujú hrabošom prezimovanie, ktoré vedie k jeho premnoženiu. Intenzívny výskyt hrabošov spôsobuje lokálne poškodenie vegetácie a vizuálne poškodenie porastu po hlodavcoch je zachytiteľné na satelitných snímkach a merateľné prostredníctvom zmien vo vegetačných indexoch (Valerio et al., 2020).

2.2.2 Pleseň zemiaková

Pleseň zemiaková je škodlivý organizmus, ktorý je zodpovedný za veľké straty na zemiakoch a paradajkách. Šíri sa vo vlhkých a mierne teplých podmienkach s vysokou vzdušnou vlhkosťou (Haverkort et al., 2012). Skorá diagnostika je dôležitá pre včasné opatrenia, pretože v priebehu niekoľkých dní dokáže zničiť celú úrodu. Detekcia ochorenia sa spolieha okrem vizuálnych zmien aj na analýzu spektrálnej odozvy napadnutých listov. Choroba mení bunkovú štruktúru a obsah chlorofylu (Fernández et al., 2020).

2.3 Diaľkový prieskum Zeme v agronómii

Diaľkový prieskum Zeme je metóda na získavanie informácií o zemskom povrchu, ktorá využíva senzory umiestnené na satelitných alebo leteckých nosičoch (Lillesand et al., 2015). V agronómii diaľkový prieskum Zeme umožňuje monitorovanie vývoja plodín, zdravia a pôdnych vlastností na obsiahlych plochách.

2.3.1 Zber satelitných dát

Konštelácia satelitov Európskej vesmírnej agentúry Sentinel-2 bola navrhnutá na poskytovanie multispektrálnych snímok s vysokým priestorovým rozlíšením, ktoré slúžia na monitorovanie zemského povrchu. Konštelácia pozostáva z dvoch identických satelitov, čo zabezpečuje vysokú frekvenciu snímania. Dáta sú navrhnuté tak, aby boli využiteľné pre služby monitorovania krajiny a potravinovej bezpečnosti. Pre analýzu vegetácie okrem základných produktov Level-1C systém umožňuje aj atmosférické korekcie na získanie odrazivosti na úrovni povrchu Level-2A (Drusch et al., 2012).

2.3.2 Vegetačné indexy

Vegetačné indexy slúžia na charakteristiku špecifických vlastností rastlín pomocou vzorcov obsahujúcich konkrétne spektrálne pásma. Hlavnú úlohu zohráva červené pásmo (RED), ktoré je silno absorbované chlorofylom pre účely fotosyntézy, a infračervené pásmo (NIR), ktoré je odrážané vnútornou bunkovou štruktúrou listov. Pre detekciu skorých štádií je významné aj Red-Edge, ktoré tvorí prechodovú oblasť medzi absorpciou a odrazom svetla (Giovos et al., 2021).

Tabuľka 1: Prehľad vegetačných indexov. Spracované podľa Giovos et al. (2021), Muruganantham (2022) a India Remote Sensing (2025).

Index	Názov	Výpočet	Popis	Využitie
NDVI	Normalized Difference Vegetation Index	$\frac{NIR - RED}{NIR + RED}$ (1)	Najpoužívanejší index zdravia rastlín.	Indikuje množstvo biomasy a chlorofylu.
NDWI	Normalized Difference Water Index	$\frac{GREEN^{1} - NIR}{GREEN + NIR}$ (2)	Monitoruje obsah vody v listoch.	Signalizuje vodný stred, ktorý môže naznačovať poškodenie koreňov.
NDMI	Normalized Difference Moisture Index	$\frac{NIR - SWIR^{2}}{NIR + SWIR}$ (3)	Zameraný na vlhkosť vnútornej štruktúry rastlín.	Detekcia vädnutia rastlín spôsobeného chorobami a suchom.

[1] Zelené pásmo
[2] Krátkovlnné infračervené žiarenie

GNDVI	Green Normalized Vegetation Index	$\frac{NIR - GREEN}{NIR + GREEN}$ (4)	Ako NDVI, ale využíva zelené pásmo namiesto červeného.	Podobne ako NDVI, ale citlivejší na obsah chlorofylu pri vysokej hustote porastu.
RECI	Red-Edge Chlorophyll Index	$\frac{NIR}{RedEdge} - 1$ (5)	Využíva Red-Edge. Vysoko korelovaný s obsahom chlorofylu a dusíka v listoch	Vhodný na detekciu žltnutia listov.

2.3.3 Meteorologické údaje

Výskyt a šírenie škodlivých organizmov závisia od podmienok prostredia a počasia. Pri predpovedi výskytu škodcov, ako je napríklad pleseň zemiaková, záleží pri rozmnožovaní na teplote a predovšetkým vlhkosti vzduchu súčasne s úhrnom zrážok, ktoré vytvárajú optimálne podmienky pre šírenie choroby (Haverkort, 2011).

Podobne aj nárast populácie hrabošov môže úzko súvisieť s počasím, konkrétne s ročnými obdobiami, vlhkosťou a množstvom zrážok (Valerio et al., 2020). Preto sú meteorologické údaje ako doplnkové informácie dôležitým aspektom v datasetoch, ktoré zvyšujú úspešnosť odhalenia rizika.

2.4 Strojové učenie

Strojové učenie je oblasťou umelej inteligencie, ktorá sa na základe algoritmov učí vykonávať odlišné úlohy. Je potrebné mať dostatok dát a vzhľadom k týmto dátam preskúmať koreláciu medzi nimi, objaviť vzory a aplikovať algoritmy, čo umožňuje získať súhrn vstupných údajov a v dôsledku určitých vzorov vytvoriť výstupy (Piuri et al., 2020). Pri definícii Toma Mitchella (1997) sa často cituje jeho formulácia:

- *„Počítačový program sa považuje za učiaci sa zo skúsenosti E vo vzťahu k istej triede úloh T a ukazovateľu výkonu P, ak sa jeho výkon pri úlohách z triedy T, meraný pomocou P, zlepšuje so skúsenosťou E.“ ~ Mitchell, 1997*

Úloha (Task), T

Úlohy strojového učenia sa zvyčajne popisujú v podmienkach ako má strojové učenie spracovávať príklad. „Príklad“ je kolekcia vlastností kvantitatívne spočítaných z konkrétneho objektu, ktorý chceme, aby systém strojového učenia spracoval. Napríklad vlastnosťami obrázka sú zväčša hodnoty pixelov na obrázku. Strojovým učením môže byť vyriešené množstvo typov úloh.

Klasifikácia je prvou z najpoužívanejších úloh, kde sa počítačový program snaží zaradiť vstupné dáta do preddefinovaných kategórií. Používa sa na rozpoznávanie objektov alebo tvárí na obrázku, kde model priradí každému obrázku príslušnú kategóriu. **Regresia** je druhou úlohou, cieľom ktorej je predikovať spojitú číselnú hodnotu na základe vstupných údajov. Na rozdiel od zaradenia vstupu do niektorej z kategórii ako je v klasifikácii, regresia vytvára funkciu, ktorá priraďuje ku každému vstupnému vektoru hodnotu z množiny reálnych čísel. Príkladom regresie je predpovedanie cien, teploty alebo výšky poistného. **Zhlukovanie** (angl. Clustering) je úlohou, pri ktorej sa vytvárajú zhluky (klastre), ktoré zoskupujú prvky s podobnými vlastnosťami (Goodfellow, Bengio and Courville, 2016).

Ukazovateľ výkonu (Performance Measure), P

Performance measure je nástroj, ktorým hodnotíme, ako vhodne model strojového učenia dokáže riešiť zadané úlohy. Kľúčovým je správny výber metriky, pretože ukazuje, ako je v praktickom nasadení model schopný napĺňať cieľ úlohy (Goodfellow, Bengio and Courville, 2016).

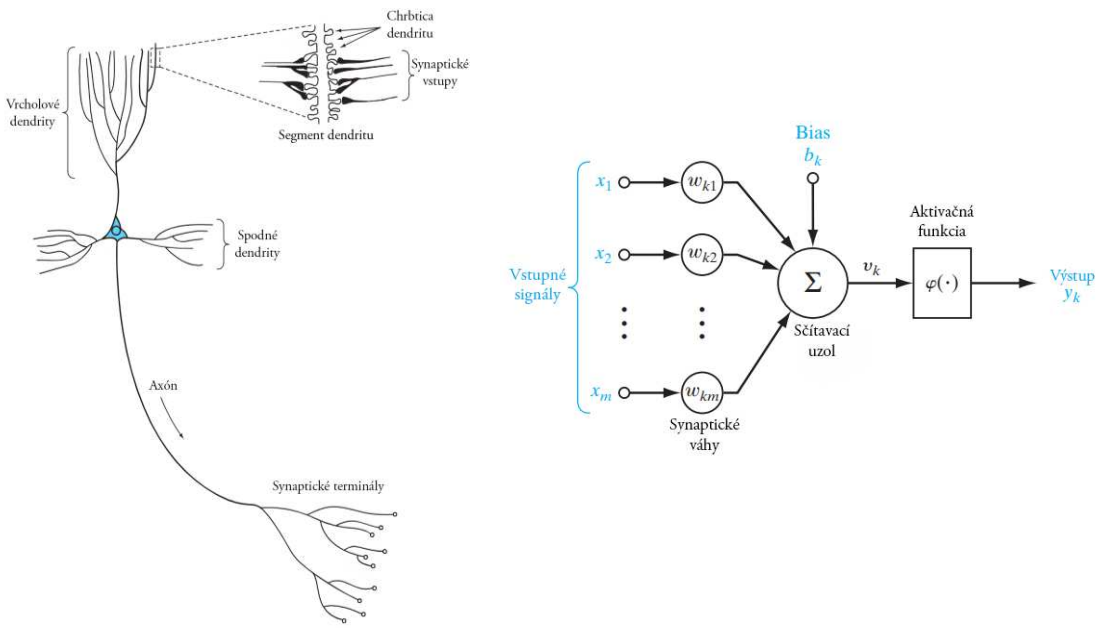
Skúsenosť (Experience), E

Algoritmy strojového učenia môžu byť kategorizované do niekoľkých podkategórií na základe skúseností, aké môžu získať počas procesu učenia. **Kontrolované (supervised)** algoritmy spracovávajú súbor označených údajov obsahujúci mnoho charakteristík a následne sa učia užitočné vlastnosti štruktúry tohoto súboru údajov. **Nekontrolované (unsupervised)** sa učia iba z neoznačených údajov bez využitia označenia relevantného pre úlohu. Po vytrénovaní je možné model doladiť pomocou označených údajov pre dosiahnutie zlepšenia generalizácie modelu (Goodfellow, Bengio and Courville, 2016).

2.5 Neurónové siete a hlboké učenie

2.5.1 Neurón a neurónová sieť

Základným stavebným kameňom pri učení umelej inteligencie je neurón. Pomocou neurónov je možné vykonávať procesy ako rozpoznávanie vzorov, vnímanie okolia alebo ovládanie motoriky človeka. Presúvajú elektrické signály z dendritov na terminály axónov cez telá axónov a takýmto spôsobom pokračujú v presúvaní po synapsii od jedného neurónu k druhému. Môžeme si dať za príklad hlístovce, ktoré majú 302 neurónov (Han et al.,2018), pričom stále prekonávajú naše počítače.



Obrázok 1: Porovnanie neurónu a umelého neurónu. Upravené podľa (Haykin, 2009).

Na základe týchto neurónov je inšpirovaná základná výpočtová jednotka neurónových sietí. Každý neurón v nej môže byť vnímaný ako samostatná funkcia, ktorá obsahuje:

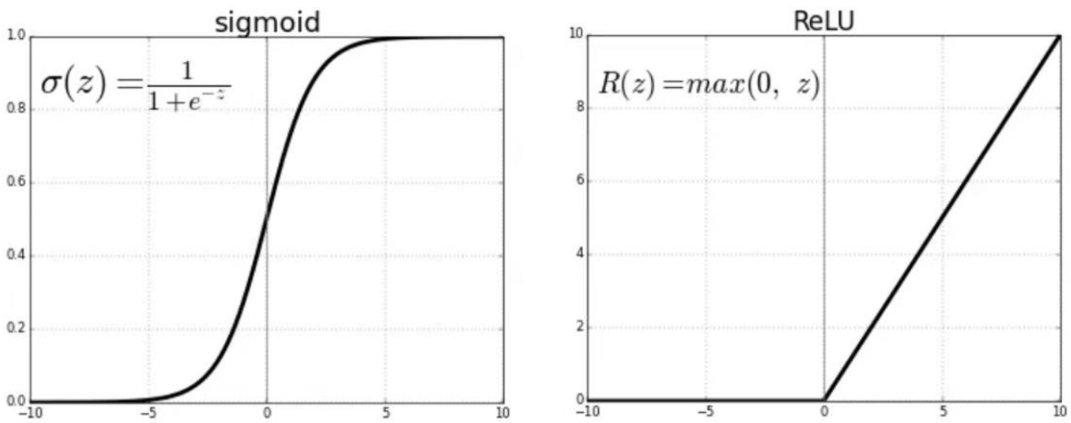
- **Vstupný vektor** — $\mathbf{x_j}$ – číselné vlastnosti vstupu
- **Váhy** — $\mathbf{w_{kj}}$ – určujú dôležitosť vstupu
- **Bias** — $\mathbf{b_k}$ – posun rozhodovacej hranice
- **Aktivačnú funkciu** — $\boldsymbol{\varphi}$ – nelineárna transformácia preaktivácie
- **Výstup** — $\mathbf{y_k}$ – hodnota po aktivácii

Vektory vstupov sa zvážia, aby sa spočítala konkrétna „váha“ daného vstupu, ku ktorej sa pričíta bias. Po pričítaní biasu sa použije aktivačná funkcia na získanie hodnôt pravdepodobnosti.

$$u_k = \sum_{j=1}^{m} w_{kj} x_j \tag{6}$$

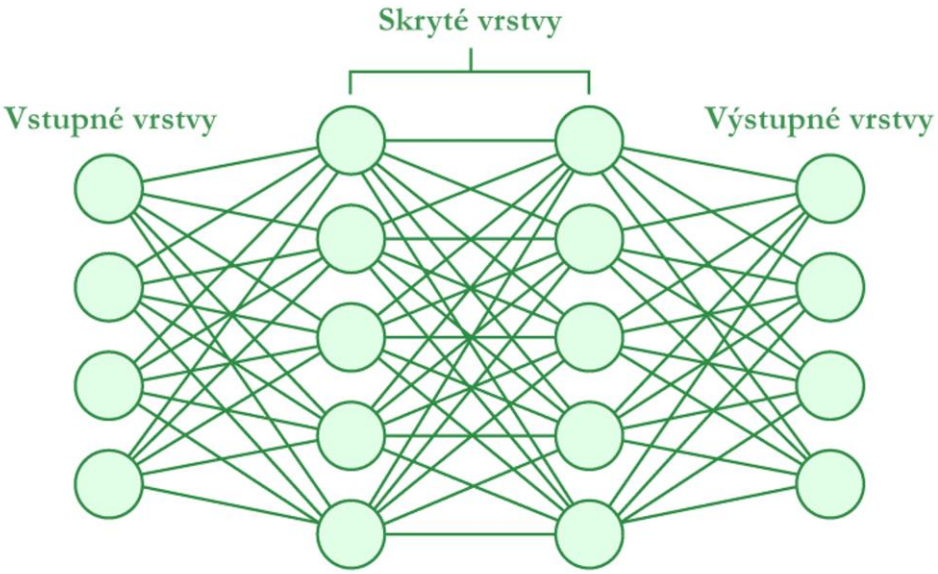
$$y_k = \varphi(u_k + b_k) \tag{7}$$

Aktivačné funkcie sú lineárne a nelineárne. Lineárne funkcie by nám nepostačili, pretože linearitou neurónová sieť vytvára výstup ako lineárnu funkciu vstupov napriek viacerým vrstvám, ktoré obsahuje. Najpoužívanejšími v súčasnosti sú ReLU a sigmoid nelineárne funkcie (Dubey et al.,2022).



Obrázok 2: Aktivačné funkcie Sigmoid a ReLU. Prevzaté z portálu Medium (2024).

Neurónová sieť je stroj navrhnutý na modelovanie spôsobom, akým ľudský mozog vykonáva určité funkcie a úlohy. Proces učenia sa vykonáva pomocou učenia algoritmu, ktorý postupne upravuje váhy tak, aby bol dosiahnutý požadovaný výkon siete. Sieť je zložená z neurónov, ktoré sú súčasťou vrstiev (Haykin, 2009).



Obrázok 3: Vrstvy neurónovej siete. Upravené z portálu GeeksForGeeks (2025).

Vstupné vrstvy sú dáta, ako napríklad obrázky, text alebo čísla, ktoré vstupujú do siete. V schovaných vrstvách každý neurón vykonáva výpočet na vstupe, ktorý posiela výsledok do ďalšej vrstvy. V každej vrstve sú dáta transformované. Po prechode cez všetky vrstvy sieť vracia konečnú predikciu, ako napríklad klasifikáciu obrázku (Han et al., 2018).

2.5.2 Proces učenia

Učenie modelu je iteratívny cyklus, ktorý sa opakuje vo viacerých epochách a jeho cieľom je maximálne znížiť chybu modelu. Pri **inicializácii** sa váhy a biasy modelu inicializujú na náhodné hodnoty, aby sa zvýšila presnosť, neuróny nemali rovnaké výpočty, vyhlo sa symetrii a gradienty mali nenulovú hodnotu (Boulila et al., 2024).

Následne prebieha **dopredná fáza**, kedy vstupný signál prechádza postupne sieťou od vstupnej vrstvy po výstupnú skrz skryté vrstvy, čím generuje predikciu modelu (Aggarwal, 2021). Táto predikcia sa porovnáva so skutočnou hodnotou za pomoci chybovej funkcie, ktorými v oblasti klasifikačných úloh sú napríklad krížová entropia (Cross Entropy) alebo MSE (Mean Squared Error – priemerná kvadratická chyba), a kvantifikuje ako veľmi sa model mýlil. Táto hodnota straty následne slúži pre prispôsobenie parametrov a zníženie chyby (Elharrouss et al., 2025).

Na základe hodnoty chyby sa použije algoritmus **spätnej propagácie**, ktorý má za cieľ vypočítať gradienty chybovej funkcie pre všetky váhy a biasy v sieti. Týmto sa určí príspevok každého parametra k celkovej chybe (Aggarwal, 2021). V poslednom kroku sa aktualizujú parametre pomocou metódy **gradientného zostupu**, kedy sa váhy posúvajú v opačnom smere ako je smer gradientu, čím sa zníži hodnota straty. Rýchlosť tohoto posunu sa definuje pomocou learning rate (Goodfellow et al., 2016). Tento cyklus sa opakuje až dokým model nedosiahne dožadovanú presnosť alebo kým nenastane konvergencia.

2.5.3 Deep learning

Podmnožina strojového učenia, ktorú nazývame hlboké učenie, sa zameriava na učenie z dát pomocou umelých neurónových sietí s viacerými vrstvami, nazývanými hlboké neurónové siete. Pred zavedením DL úlohy ako rozpoznávanie vzorov zahŕňali transformáciu surových vstupných údajov, ako sú hodnoty pixelov obrázka, na vektor vlastností, ktorý predstavuje vnútorné znázornenie údajov. Vďaka hlbokému učeniu sa tento proces dokáže vykonávať automaticky, pričom v každej skrytej vrstve sa znázornenie vstupných údajov učí alebo extrahuje hierarchickým spôsobom. Pri príklade rozpoznávania vzorov sa prvá vrstva učí prítomnosť základných prvkov ako čiary, body a hrany, druhá vrstva vzory alebo motívy rozpoznávaním kombinácií už rozpoznaných hrán, bodov a čiar a nasledujúce vrstvy kombinujú motívy, aby vytvorili sofistikovanejšie prvky, ktoré zodpovedajú vstupným dátam. Tento proces učenia prebieha v sekvencii skrytých vrstiev do bodu kedy sa vytvorí predikcia (Noor & Ige, 2025).

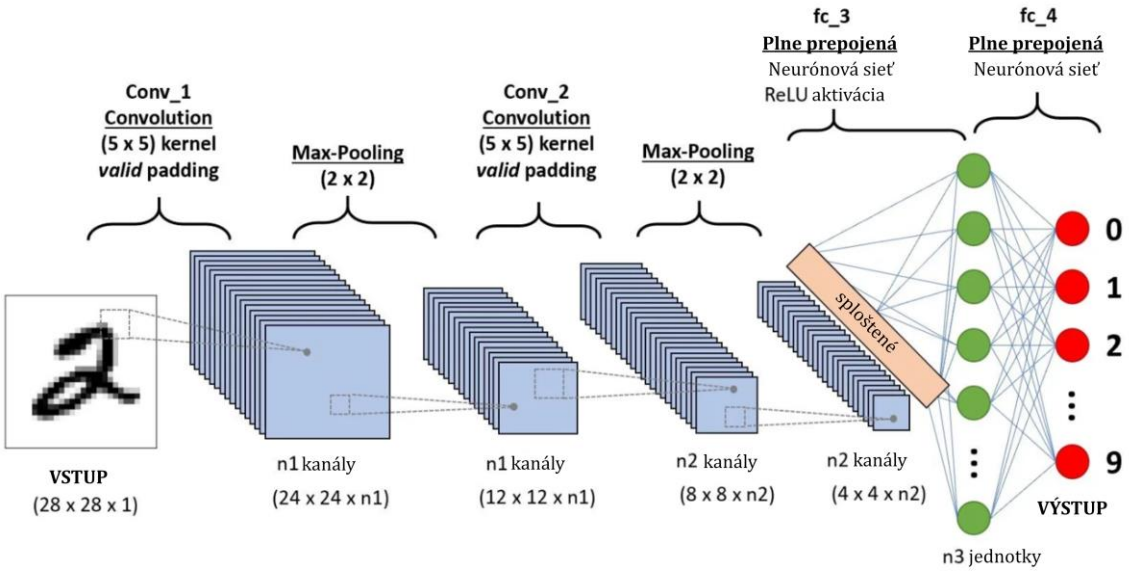
2.6 Architektúry hlbokého učenia

Dataset vhodný pre tréning v agronómii si veľakrát vyžaduje dáta rôzneho typu, ktorými môže byť text, obraz, video alebo zvuk. Tradičné modely sú často vytvorené pre jeden dátový typ, pričom pokročilé architektúry hlbokého učenia nám umožňujú fúziu[3] a analýzu rôznych zdrojov, čo vedie k vyššej presnosti pri riešení úloh (Zhang, 2023).

2.6.1 Konvolučné neurónové siete

Konvolučné neurónové siete (CNN) predstavujú triedu modelov DL, ktorá je určená pre spracovanie obrazových dát. Implementáciou konvolučných vrstiev, ktoré aplikujú kernely[4] na vstupné dáta, automaticky extrahujú priestorové črty, ako sú tvary, textúry alebo zložitejšie hrany (Krizhevsky et al., 2012). Základom architektúry CNN je konvolučná vrstva. Táto vrstva aplikuje sadu filtrov (kernelov) na vstupný obraz. Každý filter vykonáva konvolučnú operáciu, čo znamená, že sa filter posúva po vstupnej matici a takýmto spôsobom sa detegujú vzory, hrany, a následne motívy (Lecun et al., 2015). Ďalšou časťou je pooling vrstva, napríklad Max Pooling, ktorej účelom je redukcia rozlíšenia dát. Týmto znižovaním zároveň znižuje výpočtové nároky a pomáha predchádzať k pretrénovaniu. Súčasťou CNN je aj aktivačná funkcia, ktorá je v alebo na konci neurónovej siete (Kamilaris & Prenafeta-Boldú, 2018).

[3] Splynutie alebo zlúčenie. V zmysle datasetov prepojenie rôznych dátových typov.
[4] Sada trénovateľných váh



Obrázok 4: Architektúra konvolučnej neurónovej siete. Upravené z portálu Analytics Vidhya (2025).

2.6.2 Transforméry

Architektúra transformérov predstavuje hlbokú neurónovú sieť (DNN), ktorá sa zaviedla s cieľom nahradiť rekurentné a konvolučné vrstvy v sekvenčných úlohách mechanizmom pozornosti – attention (Vaswani et al., 2017). Transformér sa skladá z dvoch základných blokov, enkodéra a dekodéra.

Enkodér spracuje vstupnú sekvenciu dát a transformuje do kontextovej reprezentácie. Skladá sa z viacerých rovnakých vrstiev, ktoré obsahujú multi-hlavičkový mechanizmus pozornosti a prepojenú vrstvu. Mechanizmus pozornosti umožňuje modelu zvážiť dôležitosť každého prvku danej sekvencie vo vzťahu k ostatným prvkom a tým sa zachytávajú dlhodobé závislosti bez ohľadu na vzdialenosť prvkov v sekvencii (Vaswani et al., 2017).

Dekodér obnovuje rozlíšenie mapy príznakov na úroveň pôvodného obrazu. Prelomovou architektúrou sa stal Vision Transformer (ViT), ktorý aplikuje čistý transformér priamo na sekvencie obrazových blokov (patches). Tieto bloky sú spracovávané analogicky ako slová pri spracovaní prirodzeného jazyka, čo umožňuje modelu dosahovať lepšie výsledky pri klasifikácii obrazov (Zuo et al., 2022).

2.6.3 Transfer learning

Pri prenosovom učení sa poznatky naučené modelom pri riešení jednej úlohy využívajú ako počiatočný základ pre riešenie úlohy inej príbuznej úlohy (Yosinski et al., 2014). Hlavným cieľom je využiť už získané znalosti, aby sa dosiahlo vyššej presnosti a rýchlejšia konvergencia na menšom špecifickom datasete finálnej úlohy.

Tento mechanizmus spočíva v úprave predtrénovaného modelu, ktorý bol trénovaný na veľkom množstve vzorkov. Aplikácia obvykle obsahuje:

- Zmrazenie váh v počiatočných vrstvách.
- Doladenie (fine-tuning) váhy v hlbších vrstvách. Týmto sa znižuje výpočtová náročnosť a riziko pretrénovania na malom cieľovom datasete (Yosinski et al., 2014).

2.7 Optimalizácia a hodnotenie modelov

Úspešnosť nasadenia modelu nezáleží iba od architektúry, ale aj efektívnosti procesu jeho trénovania a validácie. Optimalizáciou učenia zabezpečíme, že model dosahuje požadovanej presnosti v prijateľnom čase. Tento proces zahŕňa voľbu správnej optimalizačnej funkcie, správne nastavenie parametrov, ako napríklad learning rate, a definovanie loss funkcie (Goodfellow et al., 2016).

Cieľom optimalizácie je minimalizácia hodnoty straty, a preto je potrebné hodnotenie modelu na trénovacích a aj validačných a testovacích dátach (Géron, 2019). Hodnotenie je uskutočnené pomocou metrík, na základe ktorých sa dokáže diagnostikovať problém, ako je pretrénovanie alebo nedostatočné prispôsobenie dátam.

2.7.1 Overfitting a underfitting

Hlavným cieľom v procese trénovania modelu DL je generálizácia, čo znamená schopnosť modelu robiť presné predikcie vrátane dát, na ktorých je natrénovaný, aj na dátach nových. Za príčinou zlyhania generalizácie môžu stáť dva hlavné problémy – overfitting a underfitting (Goodfellow et al., 2016).

Underfitting – podtrénovanie

Jav, ktorý nastáva, keď model nie je dostatočne komplexný na to, aby zachytil vzory a vzťahy prítomné v trénovacích dátach. V tomto stave je výkon modelu nízky v trénovacej a aj testovacej množine (Géron, 2019). Model sa chová akoby sa učil iba povrchové informácie. Môže dôjsť k stavom:

1. Použitie nedostatočného stavu parametrov, napríklad málo vrstiev v neurónovej sieti

2. Prišlo ku skorému prerušeniu trénovania, napríklad nízky počet epoch, kvôli tomu model nedostal dostatok času na konvergenciu. Zvýšenie kapacity model, pridaním vrstiev, neurónov alebo predĺžením tréningu je obvyklým riešením underfittingu (Goodfellow et al., 2016).

Overfitting – pretrénovanie

Stav opačný underfittingu a predstavuje najčastejší problém v DL. Nastáva, keď si model príliš detailne zapamätá šum a anomálie prítomné len v trénovacej množine, vrátane chýb pri meraní alebo unikátnych vzorov. Dôsledkom je model vykazujúci nadmerne vysokú presnosť na trénovacej množine, ale nízku presnosť na testovacej množine (Géron, 2019). Model stráca schopnosť zovšeobecňovať. Pretrénovanie je obzvlášť rizikové pri práci s nevyváženými alebo malými datasetmi.

Riešenie a regularizácia

Proti overfittingu sa využívajú techniky regularizácie, ktoré obmedzujú zložitosť modelu, čím ho nútia učiť sa všeobecnejšie vzory namiesto konkrétnych detailov dát. Medzi hlavné techniky patria dropout, čo je náhodné vypínanie určitého percenta neurónov v skrytých vrstvách a tým zabraňuje príliš veľkému spoliehaniu neurónov na iné neuróny (Srivastava et al., 2014). Druhou technikou je včasné zastavenie tréningu pri monitorovaní výkonu modelu na validačnej množine, keď sa validačná chyba začne zvyšovať, aj keď trénovacia chyba stále klesá, a tým sa model zastaví v bode pred nástupom overfittingu (Prechelt, 1998). Medzi hlavné techniky patrí ešte L1 a L2 regularizácia, čím sa myslí pridanie penalizačného člena k funkcii straty. Penalizácia obmedzuje veľkosť váh model, čo udržiava model jednoduchší a tým bráni pretrénovaniu (Srivastava et al., 2014).

2.7.2 Metriky hodnotenia

K pochopeniu metrík je dôležité si na začiatku vysvetliť maticu zámen (confusion matrix). Klasifikačná matica, následne matica zámen, ale najprv známa iba ako kontigenčná tabuľka, bola vytvorená v roku 1904 Karlom Pearsonom (Sathyanarayanan & Tantri, 2024). Riadky matice predstavujú množstvo inštancií predpokladanej triedy a stĺpce množstvo inštancií skutočnej triedy, čo nám poskytuje rozpis podľa tried presných a nepresných predikcií vykonaných klasifikátorom pre klasifikačné úlohy. True Positive (TP) nastáva, pokiaľ skutočné aj odhadované dáta sú pozitívne, True Negative (TN) sú nesprávne vyhodnotené skutočné hodnoty, pričom odhadované dáta boli taktiež negatívne vyhodnotené, pri False Positive (FP) sú predikcie pozitívne, aj keď skutočné dáta sú negatívne, a pri False Negative (FN) sú vzorky pozitívne a odhadované hodnoty negatívne.

Tabuľka 2: Matica zámen. Upravené podľa Sathyanarayanan & Tantri (2024).

		Skutočné dáta	
		Pozitívne	Negatívne
Predikcia	Pozitívne	True Positive (TP)	False Positive (FP)
	Negatívne	False Negative (FN)	True Negative (TN)

Na základe získaných hodnôt je možné evaluovať metriky. Rôzne metriky dávajú dôraz na odlišné aspekty správania algoritmu, preto je potrebné zvoliť vhodné kombinácie na každý problém. Pre klasifikačný algoritmus rozlišujeme metriky:

- **Precision** (precíznosť) – ukazuje koľko z pozorovaných dát, ktoré algoritmus predikoval ako správne, je naozaj správne vyhodnotených.

$$Precision = \frac{TP}{TP + FP} \tag{8}$$

- **Recall** (senzitivita) – ukazuje koľko skutočne správnych pozorovaných dát bolo vyhodnotených algoritmom.

$$Recall = \frac{TP}{TP + FN} \tag{9}$$

- **Accuracy** (presnosť) – určuje rozdiel správne klasifikovaných dát k celkovému počtu pozorovaných dát. Je to najviac používaná metrika a eventuálne prvá možnosť pri hodnotení výkonu algoritmu pre klasifikačné problémy (Vakili et al., 2020).

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{10}$$

- **F1-Score** (F1-skóre) – metrika, ktorá berie do úvahy precision a recall, aby na ich základe bol vypočítaný výkon algoritmu.

$$F1 - score = 2 * \frac{Precision * Recall}{Precision + Recall} \tag{11}$$

2.8 Úloha datasetu v ML

V rámci strojového učenia je základom vhodný dataset. Dataset slúži ako základ, z ktorého model získava poznatky a vzory. Kvalitou, kvantitou a štruktúrou datasetu sa priamo ovplyvňuje výkon modelu a jeho schopnosť zovšeobecňovať. V doméne agronómie a diaľkového prieskumu Zeme predstavuje tvorba adekvátnej dátovej sady výzvu, pretože agronomické dáta sú často heterogénne a citlivé na čas a priestor. Pre vývoj robustného modelu je potrebné, aby dáta reprezentovali reálne agronomické javy, minimalizovali šum a bias predtým, ako sú odovzdané do tréningového procesu (Kamilaris & Prenafeta-Boldú, 2018).

2.8.1 Štruktúra datasetov v agronómii

Agronomické datasety využívané v hlbokom učení sa vyznačujú multimodalitou, keďže pre hĺbkovú diagnostiku stavu úrody je dôležité kombinovať viacero typov vstupov (Xu & Wang, 2025). Datasety sú štruktúrované do niekoľkých kategórií na základe formátu:

1. **Obrazové dáta:** Tohto formátu sú snímky, rastrové mapy alebo iné dáta, ktoré majú usporiadanie do mriežkovej štruktúry. Tieto dáta sa spracúvajú ako tenzory a slúžia k extrakcii priestorových alebo lokálnych vzorov. Pomocou týchto dát sa dokážu merať spektrálne alebo štatistické hodnoty, ktoré kvantifikujú určité fyzikálne alebo biologické javy (Lillesand et al., 2015). Spracovanie obrazov si vyžaduje predspracovanie na odstránenie rôznych atmosférických artefaktov.

2. **Tabuľkové dáta:** Do tejto kategórie spadajú štruktúrované dáta usporiadané do tabuliek, kde stĺpce predstavujú atribúty a riadky konkrétne záznamy alebo dátové vzorky. Vstupy sú predspracované ako vektory a patria medzi nich environmentálne, geografické alebo štatistické údaje. Poskytujú kontextové informácie a často sa spájajú s obrazovými dátami pre multimodalitu datasetu (Kamilaris & Prenafeta-Boldú, 2018).
3. **Sekvenčné dáta:** Zahŕňajú vstupy s časovým aspektom a poradím prvkov. Tieto dáta umožňujú analýzu dynamiky a trendu vývoja, čo je dôležitým prvkom pre prediktívne modely (Russel & Norvig, 2021).

2.8.2 Anotácia dát

Pri procese anotácie dát (labeling) sa každému dátovému bodu priraďuje správna hodnota, čím sa vytvára ground truth – základná pravda. Tento proces je využívaný pri učení s učiteľom (Géron, 2019).

Účinnosť a presnosť modelu strojového učenia je priamo závislá od kvality anotovaných dát, na ktorých je trénovaný. Tradičné metódy manuálneho označovania sú náchylné na nekonzistentnosť a subjektivitu, čo predstavuje riziko, že model tieto chyby preberie. Ak je trénovacia množina zaťažená chybami alebo predsudkami (bias), model tieto nedostatky posilní, čo vedie k nepresným predikciám a zníženej spoľahlivosti. Pre zabezpečenie vysokej kvality je v špecifických doménach, ako je napríklad medicínska diagnostika, nevyhnutné zapojenie anotátorov[5] s expertnými znalosťami (Alamu, 2023).

2.9 Predspracovanie a čistenie

Kvalita vstupných dát určujte vhodnú mieru učenia modelu. Krokom, ktorým sa docieli kvalitného datasetu je predspracovanie dát. V reálnom svete sa vyskytujú problémy dát s neúplnosťou, šumom, nekonzistentnosťou alebo dáta obsahujú odľahlé hodnoty, ktoré môžu nesprávne ovplyvniť proces učenia. Na základe rôznych techník je cieľom dosiahnuť, aby sa surové dáta transformovali do formátu, ktorý maximalizuje schopnosť algoritmov hlbokého učenia extrahovať rôzne črty. Nedostatočné predspracovanie vedie k stavu GIGO (Garbage in, garbage out), kedy architektúra neurónovej siete zlyhá, pokiaľ vstupné informácie sú nesprávne alebo zavádzajúce (Kotsiantis et al, 2006).

2.9.1 Integrácia

Integrácia dát je proces spájania informácií z viacerých rôznych zdrojov do jednotného datasetu, ktorý je dôležitý najmä pri multimodalite, kde sa kombinujú odlišné formáty dát. Úlohou integrácie je identifikácia entít, pričom sa zabezpečí rovnaká priestorová a časová jednotka rôznych zdrojov. Efektívnou integráciou dokáže model pochopiť vzťahy medzi rôznymi typmi premenných, ktoré by pri samostatnom spracovaní neboli identifikovateľné (Han et al., 2011).

[5] Subjekt, ktorý vykonáva proces označovania (labeling)

2.9.2 Normalizácia a štandardizácia

Veľké množstvo algoritmov pri hlbokom učení je citlivé na rozsah vstupných dát. Pokiaľ sú u dát veľké rozdiely v mierkach, napríklad pri datasete na predikciu ceny domu na základe počtu izieb, ktorá bude v rozmedzí 1 až 6, a rozlohy pozemku, ktorá bude v rozmedzí 400 až 5000, model môže prisúdiť väčšiu váhu atribútom s vyššími číselnými hodnotami, čo vedie k nepresným výsledkom (Géron, 2019).

Normalizácia (Min-max scaling) – Transformuje hodnoty atribútov do fixného rozsahu, väčšinou medzi 0 a 1. Tento prístup sa používa, pokiaľ sú známe hranice hodnôt, ale je to citlivejšie na odľahlé hodnoty.

Štandardizácia (Z-score scaling)– Transformuje dáta, aby obsahovali nulovú priemernú hodnotu a jednotkovú smerodajnú odchýlku (Goodfellow et al., 2016). Táto metóda je vhodná pre dáta, ktoré sledujú Gaussovo (normálne) rozdelenie, pričom je robustnejšia voči odľahlým hodnotám oproti normalizácii.

2.9.3 Problém neúplných a nevyhovujúcich dát

Neúplné alebo nevyhovujúce dáta, ktoré sa môžu vyskytnúť zlyhaním senzorov, chybami pri prenose dát alebo prekážkami, na základe ktorých nie je možné dané dáta pozbierať. Dataset obsahujúci chýbajúce hodnoty môže priniesť problémy s výpočtom gradientov ale viesť ku skreslenému a nesprávnemu tréningu (Little & Rubin, 2019). Riešiť tieto problémy je možné niekoľkými stratégiami:

1. **Odstránenie vzoriek –** Pokiaľ množstvo chýbajúcich alebo nevyhovujúcich dát z hľadiska kvality je nízke, je možné ich vyradiť manuálne
2. **Imputácia –** Chýbajúce hodnoty sa nahradia odhadom, ktorým môže byť priemer, medián alebo výsledok komplexnejšieho prediktívneho modelu (Kotsiantis et al., 2006).
3. **Filtrácia –** Aplikujú sa algoritmy na detekciu a odstránenie nevhodných hodnôt alebo snímok. Je dôležitý korektný algoritmus, aby sa zachovali vhodné dáta a vymazali iba tie nevhodné. (Lillesand et al., 2015).

Iba zvyšovanie objemu predtrénovacích dát nemusí hneď viesť k zlepšeniu výkonu modelu, a preto sa výskum orientuje na destiláciu dát, čo je technika zameraná na redukciu rozsiahlych datasetov prostredníctvom výberu kľúčových vzoriek na zlepšenie efektivity učenia a zníženie nárokov na výpočtové zdroje (Yang et al., 2024).

2.10 Rozdelenie datasetu

Pre vyhodnotenie modelu a vyhnutie stavu pretrénovania sa musí dataset rozdeliť do viacerých množín. V rámci strojového učenia sa využíva rozdelenie na trénovaciu, validačnú a testovaciu časť.

2.10.1 Trénovacia množina

Predstavuje najväčšiu časť datasetu, zvyčajne okolo 70%, ktorú model používa priamo na učenie. Počas tohoto proces algoritmus upravuje váhy a biasy aby minimalizoval loss funkciu (Russel & Norvig, 2021).

2.10.2 Validačná množina

Táto časť slúži počas priebehu na monitorovanie modelu pri tréningu a na ladenie hyperparametrov. Model touto množinou neupravuje váhy, ale na základe jeho výkonnosti na týchto dátach rozhoduje o úpravách architektúry alebo aby sa predišlo overfittingu, o včasnom zastavení tréningu (Géron, 2019).

2.10.3 Testovacia množina

Testovacia množina slúži na zhodnotenie kvality modelu po ukončení celého tréningu a ladenia. Kvôli zavádzajúcim výsledným metrikám tieto dáta nemôžu byť použité pri tréningovej alebo validačnej fáze (Hastie et al., 2009).

2.11 Nevyváženosť tried

Nevyváženosť tried predstavuje nerovnomernú zástupnosť jednotlivých kategórií v datasete. Ako príklad môže byť vyvíjanie datasetu pre prítomnosť vzácneho ochorenia, ktoré sa vyskytuje v pomere u jedného človeka z tisíc. Dataset obsahuje údaje o 10 000 pacientoch, pričom 9 990 je zdravých a 10 pacientov je chorých. Ak by sa vytvoril jednoduchý model, ktorý má za účel predikciu zdravého človeka, tak by množina chorých ľudí bola vyhodnotená ako chyba a model by ju ignoroval. Tento nepomer spôsobuje, že klasické klasifikačné algoritmy majú tendenciu sa prikloniť smerom k väčšej triede, čím maximalizujú celkovú presnosť na stranu nesprávnej klasifikácie vzoriek z menšej kategórie (Kotsiantis et al., 2006).

Podľa Fernández et al. (2018) sa riešenie nevyváženosti dát delí na tri základné úrovne. Prvou je úroveň dát, ktorá zahŕňa techniky prevzorkovania. **Oversampling** zvyšuje počet vzoriek menšej triedy, kedy sa záznamy menšej triedy nakopírujú viackrát alebo sa vytvoria podobné syntetické dáta. **Undersampling** na druhú stranu redukuje počet vzoriek väčšej triedy, kedy sa vymazávajú vzorky pre dosiahnutie lepšieho pomeru.

Druhá úroveň je úroveň algoritmu, pri ktorej sa upravuje samotný učiaci proces, napríklad zavedením penalizácie, kedy model dostáva vyššiu pokutu za chybu pri klasifikácii menšej triedy. Treťou úrovňou je hybridný prístup, kedy sa kombinujú obidve predchádzajúce metódy s cieľom dosiahnuť najlepšiu rovnováhu (He & Garcia, 2009).

3 Metodika práce

V tejto sekcii si popíšeme metódy, ktorých využitie pomôže splniť praktickú časť bakalárskej práce. Metodika bude pozostávať z analýzy tejto oblasti, spracovania vstupných dát, návrhu a implementácie štruktúry datasetu a tréningu datasetu.

3.1 Analýza oblasti

Prístup k riešeniu vychádza z kombinácie viacerých typov dát, ktoré pozostávajú z údajov o priestore a čase výskytu agronomicky významných škodcov. Terénne záznamy poskytujú informácie o výskyte na konkrétnych miestach a v konkrétnom čase, satelitné údaje dopĺňajú informácie o stave vegetácie a povrchu súčasne so satelitnými snímkami daných polí v príslušnom čase a meteorologické údaje charakterizujú prostredie, v ktorom sa škodca vyskytuje. Výber hraboša poľného a plesne zemiakovej vychádza z ich významného dopadu na poľnohospodárstvo, vhodnosťou využitia všetkých vyššie uvedených typov údajov a taktiež z dostupnosti v databázach ÚKZÚZ ČR v posledných rokoch.

Pri návrhu metodiky bol kladený dôraz na prácu s verejne dostupnými zdrojmi dát, ich možnosť spracovania pomocou skriptov a na definíciu kľúča pre prepájanie jednotlivých modalít. Kľúčom na prepojenie je v každej forme údajov dátum, zemepisná šírka a zemepisná dĺžka.

3.2 Spracovanie vstupných dát

Základným zdrojom informácií o výskyte škodcov sú stiahnuté tabuľkové dáta z Ústředního kontrolního a zkušebního ústavu zemědělského ÚKZÚZ vo formáte CSV a období 2023 až 2024, ktoré obsahujú dátum pozorovania, geografické súradnice, identifikáciu škodlivého organizmu a triedy intenzity výskytu. Tieto dáta sú načítané v programovacom jazyku Python za pomoci knižnice pandas, kde sa zabezpečí, aby sa dátumy pozorovaní v danej oblasti neopakovali, keďže v dátach z ÚKZÚZ bolo príliš veľké množstvo opakovaní pri rovnakom dátume. Po zredukovaní a prepojení dát z rokov 2023 a 2024 sa prekonvertuje dátum do jednotného ISO formátu a súradnice sú prevedené na desatinný tvar namiesto stupňov, pričom je každému záznamu pridelený unikátny identifikátor a súbor prevedený na JSON, čo nám uľahčí neskoršiu integráciu s nasledujúcimi dátami.

Satelitné snímky a vegetačné indexy sú získané prostredníctvom Copernicus Data Space Ecosystem zo satelitu Sentinel-2 úrovne Level-2A, pričom pre každú lokalitu a dátum sú filtrované snímky v časovom intervale ±5 dní pre nájdenie najvhodnejších snímok bez oblačnosti. Pre výbere vhodných vegetačných indexov pre identifikáciu vybraných škodlivých organizmov boli použité zdroje Giovos et al. (2021), Muruganantham et al. (2022) a využitý nástroj ChatGPT ako konzultačný nástroj. V ďalšom kroku je dataset prečistený o snímky, ktoré majú stále vysokú oblačnosť alebo sú pokryté zo značnej časti snehom. S pomocou Google Earth Engine sú počítané vybrané vegetačné indexy z príslušných spektrálnych pásiem, ale snímky v Google Earth Engine nie sú dostatočnej kvality, preto tie sa zbierajú prostredníctvom CDSE.

K týmto dátam sú pre unikátne kombinácie dátumu, zemepisnej dĺžky a zemepisnej šírky získavané historické meteorologické údaje z Open-Meteo API, kde sú

z hodinových hodnôt teploty, zrážok, vetra, oblačnosti, tlaku a vlhkosti vypočítané denné štatistiky, ako sú napríklad priemery alebo súčty, ktoré sú následne pripojené s datasetom obsahujúcim už pozbierané údaje podľa spoločného kľúča.

3.3 Návrh a implementácia štruktúry datasetu

Návrh datasetu je založený na definícii jednej inštancie ako jedného pozorovania škodcu v konkrétny deň na konkrétnych súradniciach, ku ktorému sú priradené číselné atribúty zo satelitných indexov, meteorologických údajov a informácie o škodcovi a jeho výskyte. Kľúčom pre integráciu je všetkých modalít je trojica dát o dátume, zemepisnej šírke a zemepisnej dĺžke, ktorá je doplnená o unikátne ID, pričom satelitné snímky sú reprezentované cestou k uloženému PNG súboru zo Sentinel-2. Pri návrhu štruktúry sú zohľadnené požiadavky na kompatibilitu s klasickými formátmi pre tréning AI modelov, preto výstupný súbor je vo formáte CSV pre tabuľkové spracovanie a vo formáte JSON pre potreby webových a API aplikácií.

Dôležitým prvkom návrhu je multimodalita datasetu. Každá vzorka spája informácie z terénnych hlásení, spektrálnych vlastnosti zo satelitných indexov a meteorologické údaje z daného dňa na konkrétnom mieste. Takéto prepojenie umožní modelom učiť sa aj komplexnejšie vzťahy medzi stavom vegetácie, priebehom počasia a zaznamenaným výskytom škodcu alebo choroby, čo je kľúčové pre agronomické aplikácie a vhodné pre využitie pri pokročilých AI modeloch. Multimodálna štruktúra zároveň umožňuje v budúcnosti jednoducho dopĺňať ďalšie modality bez potreby zásadnej zmeny modelu dát.

Súčasťou kvalitného datasetu je systematické predspracovanie dát, ako je identifikácia a odstránenie záznamov s neúplnými súradnicami, chýbajúcimi indexmi alebo neexistujúcimi snímkami, odstránenie duplicitných záznamov na základe cesty k snímke a kontrola kvality obrazových dát prostredníctvom analýzy množstva oblačnosti alebo príliš svetlých oblastí. Meteorologické premenné a vegetačné indexy sú premenené do jednotných rozsahov napríklad pomocou min-max normalizácie na interval <0,1>, aby mali dáta porovnateľnú mierku a neobsahovali veličiny s veľkým rozsahom hodnôt.

3.4 Overenie kvality datasetu

Súčasťou tvorby tejto bakalárskej práce je aj overenie kvality vytvoreného datasetu tréningom modelu umelej inteligencie, ktorý slúži ako nástroj na posúdenie konzistencie a kvality datasetu. Dataset je rozdelený na tréningovú a testovaciu časť, prípadne rozdelený o krížovú validáciu, a na základe vybraného modelu sa vyhodnotia metriky ako presnosť klasifikácie. Tieto experimenty nesmerujú k optimalizácii najvyššieho možného výkonu, ale pre identifikáciu možných nedostatkov datasetu, ako môžu byť nevyvážené triedy intenzity, nízka variabilita niektorých atribútov alebo prítomnosť odľahlých hodnôt.

Na základe výsledkov hodnotenia budú vyhotovené odporúčania pre ďalší rozvoj datasetu, čím môže byť napríklad rozšírenie o pôdne vlastnosti alebo dátové zdroje s vyšším priestorovým rozlíšením.

4 Vlastné riešenie

V tejto časti opíšem návrh a implementáciu datasetu v agronomickom prostredí. Opíšem voľbu konkrétnych zdrojov terénnych hlásení, satelitných snímok a vegetačných hodnôt, a zdroj meteorologických údajov. Taktiež sa budem venovať krokom čistenia a filtrácie nevyhnutným pre vytvorenie kvalitného datasetu bez chýbajúcich alebo nevyhovujúcich dát. Ďalším dôležitým krokom bude normalizácia a tréning dát, ktoré popíšem následne s vyhodnotením metrík.

Preberiem konkrétne využité metódy a techniky, ktoré sú najvhodnejšie pre túto špecifickú problematiku. Cieľom bude navrhnutie robustného datasetu, ktorý bude využiteľný v praxi.

4.1 Použité nástroje a knižnice

K vypracovaniu tejto témy bolo nutné využiť knižnice navrhnuté k rozdielnym využitiam a funkciám. Popíšem využité knižnice, ktoré som využil pri zbere dát, rôznym úpravám na datasete a samotnému tréningu. Taktiež popíšem nástroje a prostredie, v ktorom som tieto knižnice využil.

Python – Programovací jazyk vhodný pre backendové aplikácie, matematiku, skriptovanie, a primárne pre túto tému, strojové učenie. Python bol prvou voľbou pri vypracovaní tohoto zadania na základe jednoduchej syntaxe a intuitívnej práce s tabuľkovými dátami. Výhodou je aj

pandas – Knižnica vyvinutá pre manipuláciu, čítanie a písanie tabuľkových dát, zlučovanie a prepájanie datasetov.

NumPy – Využívaný pre číselné výpočty a rýchlejšie spracovávanie polí.

ee – Túto knižnicu Earth Enigne využívam na zber satelitných snímok a výpočet vegetačných indexov z Google Earth Engine, ktorému sa budeme venovať v nasledujúcich kapitolách.

PIL – Obsahuje funkcie na manipuláciu s obrázkami a ich spracovanie. Táto knižnica mi pomáha pri filtrácii a čistení obrazových dát.

Google Colab – Vývojové prostredie od spoločnosti Google, v ktorom je možné využívať viac GPU, než je lokálny počítač schopný ponúknuť. Využívam ho pre tréning, pričom dataset mám nahraný na Google Drive.

scikit-learn – Knižnica napísaná v Pythone, ktorá sa používa na strojové učenie a štatistické modelovanie. Obsahuje moduly pre klasifikáciu, regresiu aj clustering a vytvorená na NumPy, SciPy a Matplotlib.

PyTorch – Open Source framework hlbokého učenia na vytváranie neurónových sietí.

4.2 Prvý zdroj dát – terénne hlásenia

Pri riešení tejto témy som si musel špecifikovať cieľ, za ktorý som vybral detekciu škodcov z dôvodu, že pri takomto zadaní je vhodné využiť všetky nutné formy dát. Oblasť som špecifikoval na polia územia Českej republiky v období rokov 2023 a 2024. Využil som všetky mesiace v roku, aby bolo možné dosiahnuť heterogenity v meteorologických údajoch.

4.2.1 ÚKZÚZ

Ústřední kontrolní a zkušební ústav zemědělský je úrad podriadený ministerstvu poľnohospodárstva, ktorý vykonáva dozor úrody a pôsobí v rastlinnolekárskej starostlivosti. Tento úrad poskytuje portál s možnosťou stiahnutia rôznych rôznych dát, pričom najväčším prínosom pre tvorbu tejto práce boli záznamy o výskyte škodcov.

Za pomoci Ing. Jakub Beránek Ph.D., ktorý mi značne pomohol s orientáciou na ich portáli a následným hlbším pochopením problematiky ohľadom škodlivých organizmov, ktoré by boli vhodné pre môj dataset, som si vybral regulovaný škodlivý organizmus pleseň zemiakovú a neregulovaného škodcu hraboša poľného. Následne bolo možné stiahnuť terénne hlásenia ohľadom uvedených dvoch škodcov pomocou jednoduchého filtrovania v obdobiach rokov 2023 a 2024. Týmto spôsobom mi vznikli prvé základné súbory *export_vyskyty2023.csv* a *export_vyskyty2024.csv*.

Tabuľka 3: Vzorové záznamy zo súboru export_vyskyty2024.csv.

Datum	2024-05-03	2024-05-28
Kraj	Středočeský kraj	Jihomoravský kraj
Okres	Praha-východ	Znojmo
Kat. území	Klíčany	Slup
Nadm. výška	278	192
GPS severní šířka	50°12’15.6″	48°47’8″
GPS východní délka	14°25’20.4″	16°12’29.7″
Latitude	50,204	48,785
Longitude	14,422	16,208
Plodina / lapač	bob obecný	brambor obecný
Škodl. org.	hraboš polní	plíseň bramboru
Vývoj. fáze ŠO	*NaN*	"napadení"
Třída výskytu	bez výskytu	bez výskytu
Intenzita	0	0

Měrná jednota	*NaN*	*NaN*
% prahu škodl.	0	0
Poznámka k metodice	"AVN/ha (březen-květen)"	"napadení rostlin"

4.2.2 Integrácia terénnych hlásení

Po zbere dát z dvoch rozdielnych rokov vznikli dve dátové sady, ktoré vzhľadom na rovnaké parametre bolo možné skopírovať do jedného súboru obsahujúceho obidve časové obdobia. Týmto som zväčšil množstvo dát na nasledujúce spracovania.

4.2.3 Zmena formátu

Terénne hlásenia boli pôvodne dostupné vo formáte CSV, ktorý vyhovuje pre tabuľkové spracovanie, ale nie je kompatibilný s rozhraním Google Earth Engine. Toto rozhranie preferuje formát JSON, a preto som musel pretransformovať základný formát súboru, aby bolo možné následné stiahnutie satelitných snímok a vegetačných indexov.

Dôležitými dátami pre extrakciu satelitných údajov boli dátum pozorovania, zemepisná šírka a zemepisná dĺžka, ktoré v datasete z ÚKZÚZ mali názvy v českom jazyku. K týmto parametrom som pridal názov škodcu a triedu výskytu pre rozoznanie škodcu súčasne s definovaním intenzity a následne premenoval do anglického znenia pre potrebu ďalšieho spracovania pomocou knižnice pandas.

```
selected_columns = ['Datum', 'Latitude', 'Longitude', 'Škodl. org.', 'Třída výskytu']
df_satellite = df[selected_columns].copy()
df_satellite.columns = ['date', 'lat', 'lon', 'pest', 'intensity']
```

Keďže pôvodný stĺpec dátumov bol v podobe českého formátu dd.mm.rrrr, musel som previesť tento formát na štandardizovanú podobu ISO 8601, ktorú GEE požaduje. Taktiež v datesete o terénnych hláseniach boli súradnice s desatinnými číslami, ktoré boli zapísané s čiarkou. Pre správnu interpretáciu v ďalších nástrojoch a knižniciach som nahradil túto desatinnú čiarku za bodku.

```
df_satellite['date'] = pd.to_datetime(df_satellite['date'], for-
mat='%d.%m.%Y').dt.strftime('%Y-%m-%d')

df_satellite['lat'] = df_satellite['lat'].astype(str).str.replace(',', '.', regex=False)
df_satellite['lon'] = df_satellite['lon'].astype(str).str.replace(',', '.', regex=False)
```

Poslednou úpravou bolo pridanie parametra identifikácie pre každý záznam kvôli vyjadreniu individuality a jednoznačnému vyznačeniu každého prvku datasetu. V takomto štádiu som mohol previesť CSV súbor na formát JSON pomocou funkcie *to_json*.

Tabuľka 4: Vzorové záznamy zo súboru satellite_locations.json.


```
{
"date":"2023-01-02",
"lat":"49.471532",
"lon":"13.266921",
"pest":"hraboš polní",
"intensity":"bez výskytu",
"id":1
},
```

```
{
"date":"2023-01-02",
"lat":"49.495226",
"lon":"12.920208",
"pest":"hraboš polní",
"intensity":"bez výskytu",
"id":2
},
```


4.3 Druhý zdroj dát – satelitné snímky a vegetačné indexy

Po vypracovaní datasetu na vhodný formát bolo možné pokročiť k zberu snímok a vegetačných indexov zo satelitných zdrojov, ktoré umožňujú hodnotiť stav porastu, keď dochádza k rozvoju škodcu.

4.3.1 Sentinel-2

Sentinel-2 poskytuje multispektrálne snímky s priestorovým a časovým rozlíšením, ktoré sa často využívajú na monitoring zdravia plodín, mapovanie chorôb a taktiež rozoznávanie poškodenia škodcami. Snímky z tohoto satelitu vrátane vegetačných indexov umožnia klasifikáciu hraboša poľného a zároveň sú vegetačné indexy dôležitými parametrami pri klasifikácii výskytu plesne zemiakovej. Výber Sentinel-2 bol na základe jeho rozlíšenia v snímkach a jeho bezplatného využitia.

4.3.2 Integrácia pomocou GEE

Dáta Sentinel-2 boli získavané prostredníctvom platformy Google Earth Engine, ktorá je prevádzkovaná v prostredí Google Cloud. Toto prostredie funguje ako služba na spracovanie geopriestorových dát. Po aktivácii účtu a vytvorení projektu s názvom identifikátoru *pest-detection-czechia* som mohol pomocou rozhrania Earth Engine API prepojiť môj skript na zber dát Sentinel-2.

```
try:
    ee.Initialize(project='pest-detection-czechia')
except Exception as e:
    ee.Authenticate()
    ee.Initialize(project='pest-detection-czechia')
```

Vstupným súborom bol dataset *satellite_locations.json*, ktorý obsahoval pre každý záznam dátum a geografické súradnice, na základe ktorých sa v GEE vyhľadali jednotlivé satelitné snímky. Sentinel-2 poskytuje dáta v hodnotách povrchovej odrazivosti, ktoré som musel vizualizovať do obrazovej formy. Využil som metódu vizualizácie v prirodzených farbách, kde boli využité pásma B4 (červená), B3 (zelená) a B2 (modrá). Hodnota odrazivosti môže dosahovať vysokých hodnôt, preto som hodnoty normalizoval v intervale 0 – 3000 a z dôvodu zvýšenia kontrastu a jasu tmavších plôch aplikoval gamma korekciu. Tento algoritmus ďalej hľadá snímky v časovom okne ± 5 dní, pokiaľ by neexistoval záznam z konkrétneho dňa, a filtrujú tie snímky, ktoré spĺňajú podmienku maximálne dvadsať percentnej oblačnosti a vyberie záznam s najvyššou čistotou pixelov.

```
        url = image.visualize(
            bands=['B4', 'B3', 'B2'],
            min=0,
            max=3000,
            gamma=1.4
        ).getThumbURL({
            'region': buffer,
            'dimensions': '256x256',
            'format': 'png'
        })

start_date = (date - timedelta(days=5)).strftime('%Y-%m-%d')
end_date = (date + timedelta(days=5)).strftime('%Y-%m-%d')

s2_collection = (ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
                 .filterBounds(point)
                 .filterDate(start_date, end_date)
                 .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))
                 .sort('CLOUDY_PIXEL_PERCENTAGE'))


        best_image = s2_collection.first()
```

Zber snímok a spektrálnych pásiem by nebol dostačujúci pre detekciu výskytu uvedených škodcov, preto boli vypočítané aj vegetačné indexy NDVI, NDWI, NDMI, GNDVI a RECI. Výhodou pri Earth Engine je, že výpočty prebiehajú v cloude a stiahnu sa iba vypočítané hodnoty.

```
# Funkcia na výpočet vegetačných indexov (normalizedDifference je vstavaná funkcia knižnice ee
na výpočet všetkých indexov okrem RECI, tú som musel vypočítať manuálne)
def calculateIndices(image):
    ndvi = image.normalizedDifference(['B8', 'B4']).rename('NDVI')
    ndwi = image.normalizedDifference(['B3', 'B8']).rename('NDWI')
    ndmi = image.normalizedDifference(['B8', 'B11']).rename('NDMI')
    gndvi = image.normalizedDifference(['B8', 'B3']).rename('GNDVI')
    reci = image.select('B5').divide(image.select('B4')).subtract(ee.Image(1)).rename('RECI')
```

Dáta získané pomocou GEE boli uložené ako nové parametre prepojené s terénnymi hláseniami výskytu škodcov a uložené do *results_complete.csv.* Snímky sa uložili do samostatného adresára *gee_data* a názvy jednotlivých obrázkov boli nazvané

ich na základe ich dátumu a geografických súradníc, a v datasete bola pre nich vytvorená cesta ako ďalší parameter. Taktiež som pridal ďalší parameter cloud_percentage pre výpis percenta oblačnosti, ktorý slúži iba ako popis pre užívateľa, na základe týchto hodnôt sa model učiť nebude.

Tabuľka 5: Vzorové záznamy súboru results_complete.csv.

id	36	5384
date	3.1.2023	11.7.2023
lat	49.620	50.659
lon	18.186	14.099
pest	hraboš polní	plíseň bramboru
intensity	bez výskytu	škodlivý výskyt
NDVI	0.291	0.848
NDWI	-0.307	-0.749
NDMI	0.041	0.384
GNDVI	0.307	0.749
RECI	0.305	1.746
red	1036.0	457.0
nir	1886.0	5553.0

date_used	2.1.2023	7.7.2023
image_path	gee_data/images/ img_49.6201_18.186307 _2023-01-03.png	gee_data/images/ img_50.658997_14.099738 _2023-07-11.png
cloud_percentage	17.316	16.471

img_49.6201_18.186307_2023-01-03.png

img_50.658997_14.099738_2023-07-11.png

Obrázok 5: Vzorové GEE snímky na základe image_path v datasete.

4.3.3 Prečistenie vysokej oblačnosti

Pri prvom spracovaní satelitných snímok bolo stále veľké množstvo nevyhovujúcich snímok obsahujúce vysokú oblačnosť alebo príliš veľký obsah snehu na obrázkoch. Takéto záznamy by boli pre učenie rušivým elementom a model by sa vytrénoval skreslene.

img_49.31957_17.34415_2024-01-17.png

img_49.397733_15.187241_2024-11-22.png

Obrázok 6: Vzorové rušivé GEE snímky

Pre riešenie tohto problému som implementoval skript *filter_cloudy_imagery.py*, ktorý analyzuje priamo stiahnuté RGB snímky na úrovni pixelov. Pre odstránenie bielych plôch som použil metódu postavenú na prahových hodnotách, ktorá transformuje obrázky do matice s pomocou knižníc Pillow a NumPy. Následne počíta percentuálne zastúpenie bielych pixelov prekračujúcich prah 240 z rozsahu 0 – 255. Pokiaľ obrázok tvorí viac ako dvadsať percent týchto pixelov, snímky sa vyfiltruje zo základného datasetu. Ďalej sa počíta priemerná hodnota všetkých pixelov v obrázku a pokiaľ je priemer vyšší ako 200, tak sa snímka považuje za preexponovanú a taktiež je vymazaná zo základnej sady. Takéto snímky sa nevymažú úplne, ale premiestnia do priečinka *removed_images*.

```
# Analýza nevhodných snímok
def analyze(image_path_str):
    img_path = Path(image_path_str)
    if not img_path.exists():
        return image_path_str, None, None, "missing"
    try:
        img = Image.open(img_path).convert("RGB")
        arr = np.array(img)
        white_px  = np.sum(np.all(arr > WHITENESS_PX_TH, axis=2))
        total_px  = arr.shape[0] * arr.shape[1]
        white_pct = (white_px / total_px) * 100.0
        brightness = arr.mean()
        if white_pct >= WHITENESS_PCT:
            status = "cloudy"
        elif brightness >= BRIGHTNESS_TH:
            status = "bright"
        else:
            status = "good"
        return image_path_str, white_pct, brightness, status
    except Exception as e:
        return image_path_str, None, None, f"error:{e}"


# Zálohovanie a odstránenie rušivých snímok
to_remove = [p for p, _, _, st in results if st in ("cloudy", "bright", "missing") or
st.startswith("error")]
for rel_path in to_remove:
    img = Path(rel_path)
    if img.exists():
        shutil.copy(img, BACKUP_DIR / img.name)
        img.unlink()
```

Ďalším dôležitým procesom algoritmu je vymazanie tých záznamov z textovej časti datasetu, ktorých snímky boli na základe image_path filtrované., aby sa dodržal rovnaký počet snímok a záznamov v CSV súbore. Učenie na datasete s neexistujúcimi snímkami by vyhadzovalo chyby, pretože by model nedokázal nájsť očakávané obrazové dáta, alebo by sa naučil na chybných dátach.

```
# Integrita dát (kontrola párov medzi záznamom a snímkou)
remaining = {str((IMAGES_DIR / f).resolve())
             for f in os.listdir(IMAGES_DIR) if f.startswith("img_")}
dataset  = {str(Path(p).resolve()) for p in df_good["image_path"]}

# Odstránime záznamy bez existujúcich snímok
df_good = df_good[df_good["image_path"].apply(lambda p: str(Path(p).resolve()) in remaining)]
```

4.4 Tretí zdroj dát – meteorologické údaje

Škodcovia sa neobjavujú náhodne a pričom satelitné dáta sú cenné informácie o stave vegetácie, pre presnejšiu detekciu výskytu škodcov sú potrebné aj dáta o podnebí na danej lokalite a v konkrétnom čase, ktoré prispievajú informačnej hodnote datasetu. Týmito dátami sa maximalizuje schopnosť učenia modelu a pridáva možnosť klasifikácie výskytu na základe väčšieho množstva parametrov.

V kontexte sledovaných škodcov sa miera vplyvu meteorologických dát líši. Pri hrabošovi poľnom sú tieto údaje doplnkom k ostatným príznakom. Aj keď hlavným znakom výskytu tohoto škodcu sú satelitné snímky a vegetačné indexy, tak dáta o počasí pomáhajú modelu zistiť vyššie riziko výskytu v danej lokalite. Na druhú stranu pri plesni zemiakovej sú tieto hodnoty dôležité pre samotné predikcie, pretože jeho životný cyklus je podmienený špecifickými meteorologickými faktormi. Rýchlosť šírenia a započatie infekcie týmto škodcom závisí od premenných ako je teplota, úhrn zrážok alebo vlhkosť vzduchu, preto sú tieto dáta pre učenie modelu dôležité.

4.4.1 Prepojenie cez Open-Meteo API

Pre zber meteorologických údajov som zvolil Open-Meteo API. Voľba vychádzala z jej bezplatného použitia, jednoduchej implementácie bez API kľúča a dostupnosti veľkého množstva kvalitných historických dát ukladaných po hodinách a zapisovaných pre konkrétnu zemepisnú šírku a dĺžku.

Pred volaním API som si vybral vhodné parametre, ktoré som chcel zbierať. Vybrané záznamy som určil podľa priameho a nepriameho vplyvu na vývoj plodín a šírenie škodcov. Tieto záznamy sú zapisované hodinovo, takže boli neskôr prepočítané na denné hodnoty.

Tabuľka 6: Zoznam parametrov a ich popis podľa Open-Meteo API, Haverkort (2011) a Valerio et al. (2020).

Názov parametra	Popis	Význam	Škodca
temperature_2m	Teplota vzduchu 2 m nad zemou	Ovplyvňuje fázy rastlín a rýchlosť množenia	Pleseň zemiaková a hraboš poľný
precipitation	Úhrn zrážok a sneh	Určuje vlhkosť pôdy, vytápanie nôr a dážď šíri spóry plesní	Pleseň zemiaková a hraboš poľný
relativehumidity_2m	Relatívna vlhkosť vzduchu	Vysoká vlhkosť spúšťa infekcie	Pleseň zemiaková
dewpoint_2m	Rosný bod	Určuje nasýtenie vzduchu a kondenzáciu vody na listoch	Pleseň zemiaková

windspeed_10m	Rýchlosť vetra	Vyvoláva prenos spór na nové lokality	Pleseň zemiaková
cloudcover	Celkové pokrytie oblačnosťou	Ovplyvňuje fotosyntézu a intenzitu UV žiarenia	Pleseň zemiaková a hraboš poľný
pressure_msl	Tlak vzduchu na hladine mora	Pomocná premenná pre určenie celkovej situácie	Pleseň zemiaková a hraboš poľný

Na zber týchto dát som použil ďalší skript písaný v Python, ktorý vyberie z CSV datasetu prečisteného o snímky s nevyhovujúcim obsahom dátum a geografické súradnice, definuje ostatné vstupné parametre a pre každú lokalitu odošle HTTP GET požiadavku, pričom využíva parametre start_date a end_date, ktoré sú zhodné s dátum pozorovania výskytu škodcu.

```
params = {
        "latitude":   lat,
        "longitude":  lon,
        "start_date": date_str,
        "end_date":   date_str,
        "hourly":     ",".join(HOURLY_PARAMS),
        "timezone":   TIMEZONE
    }
```

API vracia každý deň 24 hodinových hodnôt. Tieto dáta sa pripoja k jednému riadku v datasete a následne počíta pri teplote, vetre, oblačnosti a vlhkosti aritmetický priemer, a pri zrážkach súčet – celkový denný úhrn.

```
rec = {
        "date":            row["date"],
        "lat":             lat,
        "lon":             lon,
        "temp_mean":       df_h["temperature_2m"].mean(),
        "precip_sum":      df_h["precipitation"].sum(),
        "wind_mean":       df_h["windspeed_10m"].mean(),
        "cloud_mean":      df_h["cloudcover"].mean(),
        "pressure_mean":   df_h["pressure_msl"].mean(),
        "rh_mean":         df_h["relativehumidity_2m"].mean(),
        "dewpoint_mean":   df_h["dewpoint_2m"].mean(),
    }
```

Nakoniec sa nové meteorologické dáta spoja s pôvodným datasetom na základe unikátnych hodnôt dátumu, zemepisnej šírky a zemepisnej dĺžky, čím vznikne nový dataset, ktorý používa terénne hlásenia súčasne so satelitnými dátami a meteorologickými údajmi.

Tabuľka 7: Vzorové záznamy súboru results_enriched.csv spojené s meteorologickými údajmi.

id	10388.0	7839.0
date	13.2.2024	16.10.2023
lat	49.599	49.567
lon	18.116	16.442
pest	hraboš polní	hraboš polní
intensity	slabý výskyt	škodlivý výskyt
NDVI	0.384	0.746
NDWI	-0.387	-0.640
NDMI	0.044	0.275
GNDVI	0.387	0.640
RECI	0.434	1.143
red	840.0	617.0
nir	1886.0	4236.0
date_used	16.2.2024	12.10.2023
image_path	gee_data/images/ img_49.59953_18.116082 _2024-02-13.png	gee_data/images/ img_49.567857_16.442277 _2023-10-16.png
cloud_percentage	15.425	14.201
temp_mean	4.35	4.200
precip_sum	0.300	0.0
wind_mean	8.75	12.467
cloud_mean	77.167	34.0
pressure_mean	1016.487	1022.371
rh_mean	80.375	73.125
dewpoint_mean	1.125	-0.454

4.4.2 Problém chýbajúcich dát

Pri sťahovaní a integrácii meteorologických údajov sa vyskytli po volaní API na konkrétne miesta v danom čase prázdne hodnoty. Tento jav môže nastať, pokiaľ je absencia meraní na určitej lokalite alebo technickými problémami na strane servera.

Väčšina architektúr strojového učenia nedokáže spracovať prázdne vstupy, takže by tréningový proces zlyhal. Vybral som si metódu odstránenia nekompletných záznamov, pretože na rozdiel od imputácie, kedy by sa namiesto týchto hodnôt dopĺňal odhad, je táto metóda vhodnejšia pre spracovávanie kvalitného datasetu. Implementoval som čistiaci skript, ktorý zoberie každý záznam a zistí prítomnosť všetkých siedmich meteorologických hodnôt.

```
KEY_COLS = [
    "temp_mean",
    "precip_sum",
    "wind_mean",
    "cloud_mean",
    "pressure_mean",
    "rh_mean",
    "dewpoint_mean"
]

# Riadky s chýbajúcimi hodnotami
mask_valid   = df[KEY_COLS].notnull().all(axis=1)
df_invalid   = df[~mask_valid]
df_clean     = df[mask_valid].copy()
```

Podobne ako vo fáze filtrácie oblačnosti, aj v tomto kroku som musel zabezpečiť synchronizáciu medzi tabuľkovými dátami a satelitnými snímkami. Pokiaľ sa vymaže daný záznam zo súboru CSV, snímka iba zaberá miesto medzi ostatnými korektnými súbormi.

```
# Vymazanie obrázkov spojených s odstránenými riadkami
for img_path in df_invalid["image_path"].dropna().unique():
    p = Path(img_path)
    if not p.exists():
        p = IMAGES_DIR / p.name
    if p.exists():
        os.remove(p)
```

4.5 Riešenie nízkej kvality satelitných snímok

Po konzultácii s vedúcim práce, doc. Ing. Oldřichom Trenzom, Ph. D., sa zistilo príliš nízke rozlíšenie obrazových dát a model by sa nedokázal vytrénovať vhodne. Kvôli tomuto problému bolo nutné nahradiť zdroj snímok za alternatívny, ktorý dokáže poskytnúť prístup k satelitným snímkam vo vyššej kvalite.

Tieto nedostatky sa týkali iba obrazových dát, vegetačné indexy boli počítané priamo v prostredí GEE zo zdrojových dát Sentinel-2, preto obrázky nemajú na tieto výpočty žiadny vplyv a môžu byť zachované. Meteorologické dáta sa taktiež zachovajú zo zdroju Open-Meteo API, pretože sú nezávislou premennou a s kvalitou snímok nijako nesúvisia.

4.5.1 Nový zdroj – CDSE

Ako riešenie nedostatkov satelitných snímok som z GEE zvolil prechod na Copernicus Data Space Ecosystem. Ide o oficiálny zdroj k dátam programu Copernicus, ktorý využíva technológiu Sentinel Hub. Táto platforma umožňuje prístup k surovým satelitným dátam a ich spracovanie na strane servera bez straty kvality.

Dôvodom pre voľbu CDSE bola možnosť kontroly nad vlastnosťami výstupnej snímky. Jedná sa o API rozhranie, ktoré umožňuje definovať priestorové rozlíšenie obrázku a vlastný algoritmus na vizualizáciu. Pre zber satelitných snímok som použil skript s využitím knižnice *sentinelhub-py.* Proces zberu je rozdelený na autentifikáciu, priestorovú transformáciu a samotné generovanie obrazových dát.

Prístup k CDSE vyžaduje autentifikáciu pomocou protokolu Oauth2. Na stránke sentinelhub som si vytvoril konto, následne vygeneroval hodnoty *CLIENT_ID* a *CLIENT_SECRET*. Pomocou týchto dvoch identifikátorov bolo možné získať dočasný prístupový token. Spolu s týmto som nastavil *sh_base_url,* ktoré smeruje požiadavky na európske servery Copernicus.

```
cfg = SHConfig()
cfg.sh_client_id = CLIENT_ID
cfg.sh_client_secret = CLIENT_SECRET
cfg.sh_base_url = "https://sh.dataspace.copernicus.eu"
cfg.sh_token_url = "https://identity.dataspace.copernicus.eu/auth/realms/CDSE/protocol/openid-
connect/token"
cfg.save("cdse")
config = SHConfig("cdse")

# Overenie tokenu
tok = requests.post(
    config.sh_token_url,
    data={"grant_type": "client_credentials",
          "client_id": config.sh_client_id,
          "client_secret": config.sh_client_secret},
    timeout=30
)
```

Satelitné dáta sú primárne uložené v geografických súradniciach, ale pre moje využitie bola dôležitá fixná veľkosť a konzistentné rozlíšenie obrázka. Funkcia *point_square_bbox* rieši problém transformovaním súradníc. Zo stredu poľa, ktorým je bod o súradniciach zemepisnej šírky a dĺžky, vytvorí štvorcový výrez o reálnej veľkosti 1280 × 1280 metrov. Tento výrez sa kvôli potrebách API požiadaviek na konci vracia funkciou naspäť do podoby WGS84 – geografických súradníc počítaných v stupňov namiesto metrického systému.

```
def point_square_bbox(lat, lon, size_m):
    utm_zone = int((lon + 180) / 6) + 1

    to_utm = Transformer.from_crs(from_crs, utm_epsg, always_xy=True)
    x, y = to_utm.transform(lon, lat)

    half = size_m / 2.0
    minx, miny, maxx, maxy = x - half, y - half, x + half, y + half

    return BBox([lon_min, lat_min, lon_max, lat_max], crs=CRS.WGS84)
```

Na rozdiel od GEE, kde som sa mohol spoľahnúť na automatickú vizualizáciu hodnôt, CDSE som musel implementovať Evalscript – kód písaný v JavaScripte, ktorý sa vykoná nad surovými dátami pixelov priamo na serveri.

Tento skript definuje prevod odrazivosti na farbu:

- Vstupujú pásma červenej (B04), zelenej (B03) a modrej (B02)
- Oreže hodnoty relevantné pre vegetáciu, ktoré sú v rozsahu 0.01 – 0.35
- Aplikuje gamma korekciu kvôli tmavším snímkam

```
evalscript_png = """
const minV=0.01, maxV=0.35, gamma=1.0;
function clip(x, a, b) { return x < a ? a : (x > b ? b : x); }
function enhance(v){ return Math.pow((clip(v, minV, maxV) - minV) / (maxV - minV), 1/gamma); }
function setup() {
  return {
    input: [{ bands: ["B04","B03","B02"], units: "REFLECTANCE" }],
    output: { bands: 3, sampleType: "AUTO" }
  };
}
function evaluatePixel(s) {
  let r = enhance(s.B04);
  let g = enhance(s.B03);
  let b = enhance(s.B02);
  return [r, g, b];
}
"""
```

Snímky sú sťahované na základe parametra *date_used* pretože je vyššia šanca pre viac dostupných snímok v tomto dátume, ako z dátumu kedy bolo konkrétne pozorovanie z terénnych hlásení, keďže tieto dátumy už boli používané pri sťahovaní obrazových dát z GEE. Algoritmus načíta dátum z každého záznamu a následne žiada API o snímku presne z tohto dňa.

```
# Vytvorenie request na dátum a lokalitu
    req_png = SentinelHubRequest(
        evalscript=evalscript_png,
        input_data=[
            SentinelHubRequest.input_data(
                data_collection=S2L2A_CDSE,
                time_interval=time_interval,
                mosaicking_order=MosaickingOrder.LEAST_CC
            )
        ],
        responses=[SentinelHubRequest.output_response("default", MimeType.PNG)],
        bbox=bbox,
        size=(width, height),
        config=config
    )
```

Výsledkom tohto skriptu je tabuľkový dataset so zmenenými názvami snímok, ktoré odkazujú na nové obrazové dáta v priečinku *sentinel_data.*

S2L2A_RGB_2023-03-03_48.826319_16.128733_128x128.png

Obrázok 7: Vzorová snímka z nového zdroja CDSE

Tabuľka 8: Vzorový záznam z tabuľkovej časti datasetu prepojeného so snímkou na základe image_path.

id	2072.0
date	6.3.2023
lat	48.826
lon	16.129
pest	hraboš polní
intensity	škodlivý výskyt
ndvi	0.301
ndwi	-0.419
ndmi	-0.094
gndvi	0.419
reci	0.311
red	1532.0
nir	2850.0
date_used	3.3.2023
image_path	sentinel_data/S2L2A_RGB_2023-03-03_48.826319_16.128733_128x128.png
cloud_percentage	3.906339
temp_mean	2.0625
precip_sum	0.0
wind_mean	8.083
cloud_mean	95.333
pressure_mean	1007.854
rh_mean	65.875
dewpoint_mean	-4.071

4.6 Spracovanie pre finálnu formu datasetu

V doposiaľ pripravenej fáze datasetu sú pozbierané a navzájom integrované vhodné zdroje dát, ale ešte musí prejsť dôležitými úpravami. Tieto dáta obsahujú všetky dôležité informačné vlastnosti pre tréning modelu, ale zatiaľ predstavujú iba rôznorodú množinu údajov, ktoré by model nedokázal pochopiť.

V tejto fáze dataset pripravím do finálnej podoby na základe úprav ohľadom nevyváženosti tried, dokončenia jednoznačnej anotácie, odstránením chýbajúcich dát a rozdelením datasetu. Tieto procesy sú v dátovej vede označované ako Data Pre processing Pipeline, čo je dôležitou súčasťou tvorby datasetu a odvíja sa od nej úspešnosti pri tréningu.

4.6.1 Výskyt pretransformovaný na binárny klasifikátor

Vstupný dataset obsahoval v atribúte *intensity* rozdelenie intenzity výskytu škodcov na tri úrovne – „bez výskytu“, „slabý výskyt“ a „škodlivý výskyt“. Úrovne slabého výskytu a škodlivého výskytu bolo možné zmeniť pre jednoduchšiu úroveň „s výskytom“, ktorá dovoľuje binárnu klasifikáciu, takže jednoduchšiu úlohu pri trénovaní datasetu než multiclass klasifikácia.

Týmto krokom sa odstráni problém s hodnotením intenzity v obrazových dát a model tak dokáže jednoduchšie rozlíšiť základnú úlohu, ktorou je určiť zdravé a poškodené pole.

Pre túto transformáciu som vytvoril skript, ktorý transformuje hodnoty škodlivého a slabého výskytu na jednu množinu s výskytom. Skript najprv definuje množinu, ktoré reprezentujú prítomnosť škodcu.

```
positive_values = {
    "slabý výskyt",
    "škodlivý výskyt",
}
```

Pre stĺpec intenzity sa aplikuje funkcia na odstránenie medzier a prevod na malé písmená z dôvodu jednoduchšieho porovnávania, po ktorej sa tieto hodnoty v množine *positive_values* premapujú na názov „s výskytom“.

```
def norm_text(x: str) -> str:
    if pd.isna(x):
        return ""
    return str(x).strip().lower()

df[target_col] = df[target_col].apply(
    lambda v: "s výskytom" if norm_text(v) in positive_values else v
)
```

4.6.2 Konflikt množstva záznamov škodcov

Po dôkladnejšej analýze a za pomoci filtrovania po importe datasetu do Excelu som si všimol veľkého nepomeru tried hraboša poľného a plesne zemiakovej. Množstvo záznamov hraboša poľného bolo približne 4600 záznamov oproti 170 záznamom plesne zemiakovej, čo činilo veľkú nevyváženosť tried. Tento jav bol spôsobený zber satelitných a meteorologických dát a následnom čistení nevyhovujúcich záznamov. Pokiaľ by bol model trénovaný na týchto dátach, pleseň zemiaková by bola modelom pochopená iba ako chyba v datasete a predstavilo by to iba nadbytočný šum.

Po konzultácii ohľadom tohto problému s mojím vedúcim práce, doc. Ing. Oldřich Trenz Ph.D., rozhodol, že pleseň zemiaková bude odstránená z datasetu a model bude trénovaný iba na dátach o hrabošovi poľnom. Kvôli možnosti importu dát do prostredia Excel bolo jednoduché vyfiltrovať pleseň zemiakovú a manuálne ju z datasetu vymazať. Po odstránení moja nová dátová sada obsahovala 4670 záznamov.

4.6.3 Pripojenie merateľnej intenzity výskytu

Surové dáta čerpané z ústavu ÚKZÚZ obsahovali aj merateľné jednotky poškodenia plôch konkrétnych škodcov, ktoré by neboli použiteľné v prípade ponechania plesne zemiakovej, pretože oba škodci mali odlišné zápisy merania intenzity. Pri plesni zemiakovej sa údavala hodnota v percentách, pričom pri hrabošovi poľnom bola merateľná jednotka v počte nôr na jeden hektár. Tieto jednotky nie je možné previesť na rovnakú merateľnú jednotku a z tohto dôvodu nemohli byť pridané do datasetu.

V situácii datasetu tvoreného iba hrabošom poľným tieto užitočné informácie sú využiteľné. Táto jednotka pomôže modelu presnejšie určiť výskyt, čím zvyšuje kvalitu celkového datasetu. Riešením implementácie bol skript, ktorý pridá tieto hodnoty do nového stĺpca s názvom *intensity_nor_ha* hneď po anotácii v stĺpci *intensity*. Výsledkom je dataset obsahujúci konkrétne hodnoty výskytu hraboša poľného súčasne s binárnou klasifikáciu.

```
merged_df = pd.merge(
        df_results,
        df_vyskyty_subset[['date', 'lat_round', 'lon_round', 'pest', 'intensity_nor_ha']],
        on=['date', 'lat_round', 'lon_round', 'pest'],
        how='left'
    )
```

4.6.4 Kontrola integrity

Následkom toho, že satelitné snímky a tabuľkové dáta boli spracovávané v oddelených procesoch, došlo k nekonzistentnosti medzi počtom záznamov v CSV súbore a počtom obrázkov v priečinku. Bolo nutné zabezpečiť, aby každý riadok v textovom datasete mal iba jednu satelitnú snímku a ostatné dáta boli vymazané.

Proces kontroly fungoval na skripte, ktorý kontroluje stĺpec *image_path* a či cesty k obrázkom reálne odkazujú na existujúce obrázky, a taktiež kontroluje validitu snímky a či daný obrázok nie je poškodený na základe veľkosti obrázka, takže či jeho veľkosť nie je 0 bajtov.

```
for index, row in tqdm(df.iterrows(), total=df.shape[0]):
        img_path = row['image_path']

        if not os.path.exists(img_path):
            missing_count += 1
            continue

        try:
            with Image.open(img_path) as img:
                img.verify()

            valid_indices.append(index)

        except (IOError, SyntaxError) as e:
            corrupt_count += 1
            continue
```

Po dokončení a prečistení datasetu skriptom CSV súbor obsahoval o 11 záznamov viac ako priečinok obrazových dát. Tento problém nastal, keď boli dve pozorovania vykonané v krátkom časovom rozostupe na rovnakých územiach, takže algoritmus na zber dát z CDSE pri výbere najlepšej snímky priradil obom záznamom tú istú snímku. Pokiaľ by tieto hodnoty zostali v datasete, mohlo by to viesť k skresleniu trénovania. Z dôvodu malého množstva použitých rovnakých snímok som na analýzu využil umelú inteligenciu, ktorá analyzovala CSV súbor a vytvorila zoznam riadkov používaných rovnaký *image_path*. Na základe tejto tabuľky som porovnal medzi sebou dané záznamy a vymazal záznam, ktorý mal väčší rozdiel medzi *date* a *date_used.* Týmto vznikol dataset, ktorý má rovnaké množstvo záznamov a priečinok s rovnakým množstvom satelitných snímok, kedy riadok odkazuje na jednu konkrétnu snímku.

4.6.5 0/1 anotácia

Poslednou transformáciou pred finálnym filtrovaním bola zmena hodnôt v stĺpci *intensity* z textovej podoby na číselnú. Modely strojového učenia pracujú iba s číselnými tenzormi, textové reťazce sú primárne pre ľudské pochopenie, preto bola táto transformácia dôležitou súčasťou.

Vytvorený skript menil hodnoty atribútov „bez výskytu“ na číselnú hodnotu 0 a hodnoty „s výskytom“ na 1. Taktiež bol pripojený parameter *intensity_text* na koniec datasetu. Tento parameter nebude vstupovať pri tréningu na datasete, jeho zmysel je iba pre orientáciu užívateľa a jeho porozumenie konkrétnym záznamom.

```
    # Vytvorenie kópie
    df['intensity_text'] = df['intensity']

    mapping = {
        'bez výskytu': 0,
        's výskytom': 1,
    }

    # Transformácia
    df['intensity'] = df['intensity'].map(mapping)
```

Tabuľka 9: Finálne parametre textovej časti datasetu.

id	86
date	17.1.2023
lat	50.204036
lon	17.209465
pest	hraboš polní
intensity	1
intensity_nor_ha	3800
ndvi	0.6322999279019467
ndwi	-0.6016979129819596
ndmi	0.1595390524967989
gndvi	0.6016979129819596
reci	0.984313725490196
red	510.0
nir	2264.0
date_used	17.1.2023
image_path	sentinel_data/S2L2A_RGB_2023-01-17_50.204036_17.209465_128x128.png
cloud_percentage	15.825388
temp_mean	0.7916666666666666
precip_sum	0.5
wind_mean	12.991666666666667
cloud_mean	88.95833333333333
pressure_mean	990.4375
rh_mean	83.75

dewpoint_mean	-1.7375
intensity_text	s výskytom

4.6.6 Filtrovanie oblačnosti snímok CDSE

Po prechode na zber satelitných snímok zo zdroja CDSE aj napriek podmienkam v hlavnom skripte sťahovania dát sa stále v skripte vyskytovali snímky, s ktorými som nebol spokojný. Dataset som chcel posunúť do stavu maximálnej možnej kvality, tak som použil skript napísaný pre dodatočné vyčistenie nadmernej oblačnosti alebo snežného pokrytia na základe bielych pixelov, príliš veľkého obsahu jasu na obrázkoch a objavili sa aj snímky s čiernymi oblasťami, ktoré taktiež predstavujú šum pre tréning.

Na začiatku som zvolil prahové hodnoty, ktoré boli nastavené a menené na základe množiny výstupu, aby sa minimalizoval počet nevyhovujúcich výsledkov.

```
CLOUD_META_TH   = 10.0    # Maximálne povolené % oblačnosti v metadátach
WHITENESS_PX_TH = 190     # Prahová hodnota pre jas pixelu
WHITENESS_PCT   = 5.0     # Maximálne povolené % bielych pixelov
BRIGHTNESS_TH   = 160.0   # Maximálny povolený priemerný jas fotky
BLACK_PCT_TH    = 10.0    # Maximálne povolené % čiernych pixelov
```

Logika analýzy bola vo funkcii *analyze_image_quality*, ktorá spracuje obraz ako trojrozmerné pole.

```
def analyze_image_quality(row_data):
    try:
        with Image.open(img_path) as img:
            arr = np.array(img.convert("RGB"))
            total_px = arr.shape[0] * arr.shape[1]

            # Čierne pixely
            black_px = np.sum(np.all(arr == 0, axis=2))
            black_pct = (black_px / total_px) * 100.0
            if black_pct > BLACK_PCT_TH:
                return idx, "bad", f"Black Edges {black_pct:.1f}%"

            # Príliš jasné snímky
            avg_brightness = np.mean(arr)
            if avg_brightness > BRIGHTNESS_TH:
                return idx, "bad", f"Too Bright Avg {avg_brightness:.1f}"

            # Biele pixely
            white_mask = np.all(arr > WHITENESS_PX_TH, axis=2)
            white_px = np.sum(white_mask)
            white_pct = (white_px / total_px) * 100.0

            if white_pct > WHITENESS_PCT:
                return idx, "bad", f"White Snow {white_pct:.1f}%"
```

Kvôli výpočtovej náročnosti otvárania všetky obrázkov bola využitá knižnice *concurrent.futures* a jej modul *ThreadPoolExecutor*. Pomocou tejto metódy je možný paralelný beh analýzy viacerých jadier procesora, čím sa čas spracovania zredukoval z minút na sekundy.

Nevyhovujúce snímky sú odstránené z priečinka vhodných obrazových dát a súčasne vymazané ich dáta z CSV súboru na základe *image_path.*

4.6.7 Rozdelenie datasetu na Train/Val/Test

Poslednou fázou celkového predspracovania datasetu je rozdelenie na časti trénovacej, validačnej a testovacej množiny. Toto rozdelenie slúži ako downsampling veľkého datasetu, kedy som pomerovo spracoval dáta na 70% trénovacej množiny, keďže je najdôležitejšou časťou na ktorej sa model bude trénovať, 15% validačnej a 15% testovacej množiny. Pre vytvorenie súborov s rovnakým pomerovým zastúpením záznamov ako v základnom datasete sa použila technika stratifikovaného výberu, kedy sa zachová rovnaký pomer výskytu škodcov, dátumov a oblastí. To znamená, že pokiaľ bol napríklad pomer „s výskytom“ a „bez výskytu“ 3:2, tak by sa tento pomer zachoval aj v trénovacej, validačnej a testovacej množine.

```
# Oddelenie testovacej množiny
    # stratify zabezpečí pomer
    df_train_val, df_test = train_test_split(
        df,
        test_size=TEST_SIZE,
        random_state=RANDOM_SEED,
        stratify=df['intensity']
    )

# Rozdelenie na trénovaciu a validačnú množinu
    relative_val_size = VAL_SIZE / (1 - TEST_SIZE)

    df_train, df_val = train_test_split(
        df_train_val,
        test_size=relative_val_size,
        random_state=RANDOM_SEED,
        stratify=df_train_val['intensity']
    )
```

Po aplikácii algoritmu na dataset mi vznikli tri súbory, *data_train.csv*, *data_val.csv* a *data_test.csv*. Dataset obsahuje terénne hlásenia, satelitné dáta a meteorologické údaje, ktoré sú prepojené na základne unikátnych kombinácii dátumov a geografických súradníc, očistený o nevyhovujúce záznamy, anotovaný, vyvážený a rozdelený na hlavné množiny pre tréning.

4.7 Učenie

Na spracovanom datasete kombinovanom obrazovú a tabuľkovú zložku, som zvolil prístup multimodálneho strojového učenia. Cieľom vytrénovania modelu na datasete je klasifikácia zdravého alebo poškodeného poľa na základe výskytu škodcu podľa daného záznamu. Vybral som konvolučnú neurónovú sieť na obrazové dáta a viacvrstvový perceptrón na učenie pomocou číselných atribútov.

4.7.1 ResNet

Architektúra konvolučných neurónových sietí, ktorá už bola predučená na veľkom množstve dát. Vďaka tejto architektúre je samotné učenie rýchlejšie, pretože model už rozumie základným vzorom v obrázkoch a nemusí riešiť každú maličkosť a na novo sa učiť všetky základy. Vďaka využitiu reziduálnych blokov dokáže sieť eliminovať problém miznúce gradientu, čím sa váhy v sieti optimalizujú rýchlejšie už od prvých epoch (He et al., 2015).

Túto architektúru som vybral kvôli nízkej pamäťovej náročnosti v porovnaní s inými hlbokými sieťami, čo umožnilo trénovanie aj v prostredí s limitovanou GPU pamäťou. Ďalšou výhodou, spojenou s nízkou pamäťovou náročnosťou, bola rýchlosť učenia a konvergencie modelu.

4.7.2 MLP

Architektúra dopredných (feedforward) neurónových sietí, ktorá obsahuje minimálne tri vrstvy neurónov, vstupnú, minimálne jednu skrytú a výstupnú vrstvu. Okrem vstupných neurónov každý neurón používa nelineárnu aktivačnú funkciu, čo umožní modelovať zložité nelineárne vzťahy v dátach. Učenie prebieha metódou supervised learning pomocou spätnej propagácie, ktorý iteratívne upravuje váhy na základe rozdielu predikovaného a skutočného výstupu (Popescu et al., 2009).

Pre spracovanie tabuľkovej časti datasetu som vybral MLP z dôvodu nelinearity, ktorá je potrebná pri dátach rôznych typov, v mojom prípade napríklad biologických a klimatických. Pomocou skrytých vrstiev a nelineárnych aktivačných funkcií sieť dokáže aproximovať zložité vzťahy medzi vstupnými vektormi a výstupom.

4.7.3 Proces učenia

Tréning modelu sa odohrával v prostredí Google Colab, primárne kvôli bezplatnému prístupu k hardvérovej akcelerácii pomocou NVIDIA T4 GPU, čo urýchlilo tréning konvolučných sietí. Bez tohto využitia bezplatných GPU by trénovanie mohlo trvať exponenciálne dlhšie.

```
# Nastavenie zariadenia
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

1. Načítanie dát

Aby mal Google Colab prístup k dátam som skomprimované obrazové dáta súčasne so súbormi *data_train.csv*, *data_val.csv* a *data_test.csv* uložil do svojho cloudové účtu na Google Drive, z ktorého ich bolo možné importovať do prosredia Google Colab.

```
drive.mount('/content/drive')

DRIVE_FOLDER = '/content/drive/MyDrive/Bakalarka_data'
LOCAL_DIR = '/content'

if not os.path.exists(DRIVE_FOLDER):
    print(f"Priečinok '{DRIVE_FOLDER}' neexistuje.")
else:
    shutil.copy(f"{DRIVE_FOLDER}/data_train.csv", LOCAL_DIR)
    shutil.copy(f"{DRIVE_FOLDER}/data_val.csv", LOCAL_DIR)
    shutil.copy(f"{DRIVE_FOLDER}/data_test.csv", LOCAL_DIR)

    # Rozbalenie obrázkov
    zip_path = f"{DRIVE_FOLDER}/sentinel_data.zip"
    os.system(f"unzip -q -o '{zip_path}' -d '{LOCAL_DIR}'")
```

2. Data Pipeline

Definoval som triedu *VoleDataset*, ktorá slúži ako most medzi súbormi na disku a neurónovou sieťou. V každej iterácii naraz načíta satelitnú snímku a súčasne aj jej údaje z CSV súboru. Táto trieda obsahuje konštruktor *__init__* a metódu *__getitem__*, ktorá zabezpečí načítanie snímok z disku a konverziu do RGB. Následne metóda použije augmentačné transformácie a vráti tenzor obrázku, tenzor tabuľkových dát a anotáciu.

```
class VoleDataset(Dataset):
        # Načítanie tabuľkových dát
    def __init__(self, csv_file, img_dir, tabular_cols, transform=None, scaler=None):
            self.tabular_data = self.df[tabular_cols].values.astype(np.float32)
            if scaler:
                self.tabular_data = scaler.transform(self.tabular_data)

            self.labels = self.df['intensity'].values.astype(np.float32)
            self.img_paths = self.df['image_path'].values

        # Načítanie snímky
        def __getitem__(self, idx):
            img_name = os.path.basename(self.img_paths[idx])
            img_path = os.path.join(self.img_dir, img_name)
            image = Image.open(img_path).convert("RGB")

            if self.transform:
                image = self.transform(image)

            return image, torch.tensor(self.tabular_data[idx]),
                   torch.tensor(self.labels[idx]).unsqueeze(0)
```

Pre trénovaciu množinu bola použitá augmentácia dát, čo zvyšuje rozdielnosť vstupov a zvýši ochranu proti overfittingu, s ktorým som mal na začiatku problém a snažil sa ho minimalizovať. Tento postup je na základe Kumar et al. (2020), pretože zvyšovanie objemu trénovacích dát je často nevyhnutné na zníženie pretrénovania a zvýšenie robustnosti modelov strojového učenia v úlohách s nedostatkom dát. Do toho patria transformácie ako vertikálne a horizontálne preklopenie alebo rotácia o 30 stupňov. Taktiež bola pridaná transformácia na základe zmeny jasu alebo kontrastu. Tieto úpravy simulujú podmienky z reálneho života, ktoré by pre model mohli byť počas testovania nejasné. Po týchto zmenách následuje konverzia dát, pričom je každá snímka upravená na rozlíšenie 224 × 224 pixelov, ktoré sú vyžadované architektúrou ResNet.

Snímka je prevedená na tenzor, ktorým sú hodnoty pixelov preškálované z rozsahu 0–255 na interval <0;1>. Nakoniec sa normalizuje tenzor pomocou tabuľkových hodnôt priemeru a smerodajnej odchýlky datasetu ImageNet, čo zabezpečí stabilitu gradientov v priebehu učenia.

Pre validačnú a testovaciu množinu som vynechal kroky augmentácie, aby hodnotenie modelu bolo čisto na vytvorenom datasete. Pre tieto dáta boli použité zmeny veľkosti, konverzia na tenzor a normalizácia.

```
# Trénovacia množina
train_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(30),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# Validačná a testovacia množina
val_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

Pred vytvorením datasetov sa inicializuje normalizácia pomocou *StandardScaler*, ktorej parametre sú vypočítané na základe trénovacej množiny. Týmto sa zabráni úniku dát a štatistické vlastnosti testovacích dát neovplyvnia tréning.

Trieda *VoleDataset* prevezme *scaler* pre všetky tri množiny. Na záver sa datasety zapuzdria do objektov *DataLoader*, ktoré zabezpečia načítavanie dát do pamäte po častiach.

```
train_df_temp = pd.read_csv(TRAIN_CSV, sep=';', encoding='cp1250')
scaler = StandardScaler()
scaler.fit(train_df_temp[TABULAR_COLUMNS].values)

# Vytvorenie Datasetov a DataLoaderov
BATCH_SIZE = 32

train_dataset = VoleDataset(TRAIN_CSV, IMG_DIR, TABULAR_COLUMNS, train_transforms, scaler)
val_dataset   = VoleDataset(VAL_CSV, IMG_DIR, TABULAR_COLUMNS, val_transforms, scaler)
test_dataset  = VoleDataset(TEST_CSV, IMG_DIR, TABULAR_COLUMNS, val_transforms, scaler)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader   = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader  = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)
```

3. Architektúry a ich fúzia

Jadrom návrhu pre riešenie tejto problematiky je trieda *MultimodalResNet*, ktorá implementuje a integruje dve rozdielne neurónové siete do jedného celku. Pre satelitné snímky som použil architektúru ResNet-18, pri ktorej je model inicializovaný s predtrénovanými váhami na datasete ImageNet, ktorá aplikuje transfer learning. Keďže je môj dataset v porovnaní s ImageNet malý, bolo implementované čiastočné zmrazenie váh. Cyklus prechádza bloky siete a pre prvých 5 blokov nastaví, že príznaky pre detekciu hrán, rohov a základných textúr ostanú nezmenené, zatiaľ čo sa budú učiť iba špecifickejšie príznaky na detekciu hraboša.

Pre dáta z CSV súborov implementujem viacvrstvový perceptrón pomocou kontajnera *nn.Sequential*. Vstupný vektor je transformovaný na skrytú vrstvu so 64 neurónmi a následne komprimovaný do výstupného vektora s veľkosťou 32 neurónov. Pre zachytenie komplexnejších vzťahov medzi jednotlivými dátami sú medzi lineárne vrstvy vložené nelineárne aktivačné funkcie ReLU. Pre problémy s overfittingom som pridal vrstvu Dropout, ktorá náhodne uspáva polovicu neurónov v priebehu tréningu a tým zabráni, aby sa model príliš spoliehal na konkrétne vstupy.

Tretia časť spája informácie oboch architektúr. Vstup je vektor s veľkosťou súčtu výstupov z CNN(512) a MLP(32), ktorý je privedený do finálneho klasifikátora, ním redukovaný na 128 neurónov a následne zredukovaný až na jediný výstupný neurón.

Poslednou implementovanou metódou je metóda *forward* definujúca tok dát sieťou počas tréningu, ktorá prebehne v troch krokoch:

I. Satelitné snímky prechádzajú cez CNN, čím sa získa vektor *x_img*
II. Tabuľkové dáta prechádzajú cez MLP, čím sa získa vektor *x_tab*
III. Vektory sa spoja do *x_combined*. Tento vektor obsahuje plnú informáciu o skúmanom poli a vstúpi do finálneho klasifikátora, ktorý vráti predikciu

```
class MultimodalResNet(nn.Module):
    def __init__(self, num_tabular_features):
        super(MultimodalResNet, self).__init__()

        # ResNet
        self.cnn = models.resnet18(pretrained=True)
```

```
        ct = 0
        for child in self.cnn.children():
            ct += 1
            if ct < 6:
                for param in child.parameters():
                    param.requires_grad = False

        num_cnn_features = self.cnn.fc.in_features
        self.cnn.fc = nn.Identity()


        # MLP
        self.tabular_net = nn.Sequential(
            nn.Linear(num_tabular_features, 64),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(64, 32),
            nn.ReLU()
        )

        # Fúzia
        self.final_classifier = nn.Sequential(
            nn.Linear(num_cnn_features + 32, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, 1)
        )

    def forward(self, image, tab_data):
        x_img = self.cnn(image)
        x_tab = self.tabular_net(tab_data)
        x_combined = torch.cat((x_img, x_tab), dim=1)
        output = self.final_classifier(x_combined)
        return output
```

Po definícii triedy *MultimodalResNet* sa model inicializuje a nastavia sa parametre pre tréning. Pre výpočet chyby bola použitá stratová funkcia pre binárnu klasifikáciu a pre aktualizáciu váh som použil optimalizačný algoritmus Adam. Výhodou pri tomto algoritme bolo prispôsobovanie rýchlosti učenia pomocou zápisu konkrétnej rýchlosti learning rate a pre boj s overfittingom bol pridaný parameter weight decay, ktorý núti model hľadať jednoduchšie vzory v dátach. Na konci je pridaný plánovač rýchlosti učenia, ktorý sleduje vývoj validačnej chyby. Pokiaľ sa model začne zhoršovať, learning rate sa automaticky zníži na základe *patience*, kde sa zapisuje počet prejdených zhoršujúcich sa epoch pre zníženie learning rate.

```
model = MultimodalResNet(num_tabular_features=len(TABULAR_COLUMNS)).to(DEVICE)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=0.00001, weight_decay=1e-4)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', patience=1)
```

4. Proces tréningu

Proces tréningu sa využíva na optimalizáciu parametrov neurónov siete. Prebieha cyklicky po dobu maximálne v 20 epoch, pričom v každej epoche je tréningová a validačná fáza. V tréningovej fáze prepínam model do režimu pomocou *model.train()*. V tomto kroku sa aktivujú vrstvy a iteruje cez dávky (batches) dát na trénovanie. Dáta sa presúvajú na GPU a následne vykonáva backpropagation.

```
model.train()


for images, tab_data, labels in loop:
        images, tab_data, labels = images.to(DEVICE), tab_data.to(DEVICE), labels.to(DEVICE)

        optimizer.zero_grad() # Vynulovanie gradientov
        outputs = model(images, tab_data) # Dopredný prechod
        loss = criterion(outputs, labels) # Výpočet chyby
        loss.backward() # Spätná propagácia
        optimizer.step() # Aktualizácia váh
```

Po prechode tréningovými dátami pokračuje validácia, na ktorú sa prepína pomocou *model.eval()* a pre zníženie spotreby pamäte súčasne so zrýchlením vyhodnotenia na neznámych dátach pre model sa použije *torch.no_grad()*.

```
model.eval()


with torch.no_grad():
        for images, tab_data, labels in val_loop:
            images, tab_data, labels = images.to(DEVICE), tab_data.to(DEVICE), labels.to(DEVICE)
            outputs = model(images, tab_data)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
```

U každej epochy sa na konci použije Learning Rate Scheduler, čo je mechanizmus sledujúci priemernú validačnú stratu a pri stagnácii zníži rýchlosť učenia, čím model dokáže vhodnejšie dolaďovať vo finálnych fázach.

```
scheduler.step(avg_val_loss)
```

Pri tréningu sa vyskytoval problém overfittingu už pri druhej epoche. Preto som pridal skoré zastavenie, ktoré ukladá model s najlepšími výsledkami na validačnej vrstve namiesto toho, aby bol model ukladaný automaticky po každej epoche. Tréning sa predčasne zastaví, pokiaľ sa model nezlepší nad aktuálne najoptimálnejšie vytrénovanie počas ďalších ôsmich epochách.

```
if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        torch.save(model.state_dict(), 'model.pth')
        print(f"Uloženie")
        patience_counter = 0
    else:
        patience_counter += 1
        print(f"Žiadne zlepšenie ({patience_counter}/{EARLY_STOP_LIMIT})")

    if patience_counter >= EARLY_STOP_LIMIT:
        print("\nZastavený tréning")
        break
```

4.8 Vyhodnotenie

Záverečným krokom pri spracovaní datasetu a tréningu modelu je vyhodnotenie výkonu natrénovaného modelu. Pre overenie presnosti som využil testovaciu množinu dát, ktorá nebola použitá ani v tréningovej a ani vo validačnej fáze.

Model sa prepne do režimu *model.eval()* a prejde testovací dataset. Výstupom sú logity[6], ktoré sa transformujú na pravdepodobnosť v intervale <0;1>. Binárna klasifikácia sa vytvorí pridaním prahovej hodnoty 0.5, kedy hodnoty nad túto úroveň sú klasifikované ako výskyt hraboš a hodnoty pod 0.5 ako bez výskytu.

```
model.eval()


with torch.no_grad():
    for images, tab_data, labels in test_loader:
        images, tab_data, labels = images.to(DEVICE), tab_data.to(DEVICE), labels.to(DEVICE)
        outputs = model(images, tab_data)
        preds = torch.sigmoid(outputs) > 0.5
```

Ďalším krokom je výpočet metrík, ktoré boli vypočítané pomocou knižnice *sklearn.metrics*.

```
acc = accuracy_score(all_labels, all_preds)
prec = precision_score(all_labels, all_preds)
rec = recall_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds)
cm = confusion_matrix(all_labels, all_preds)
```

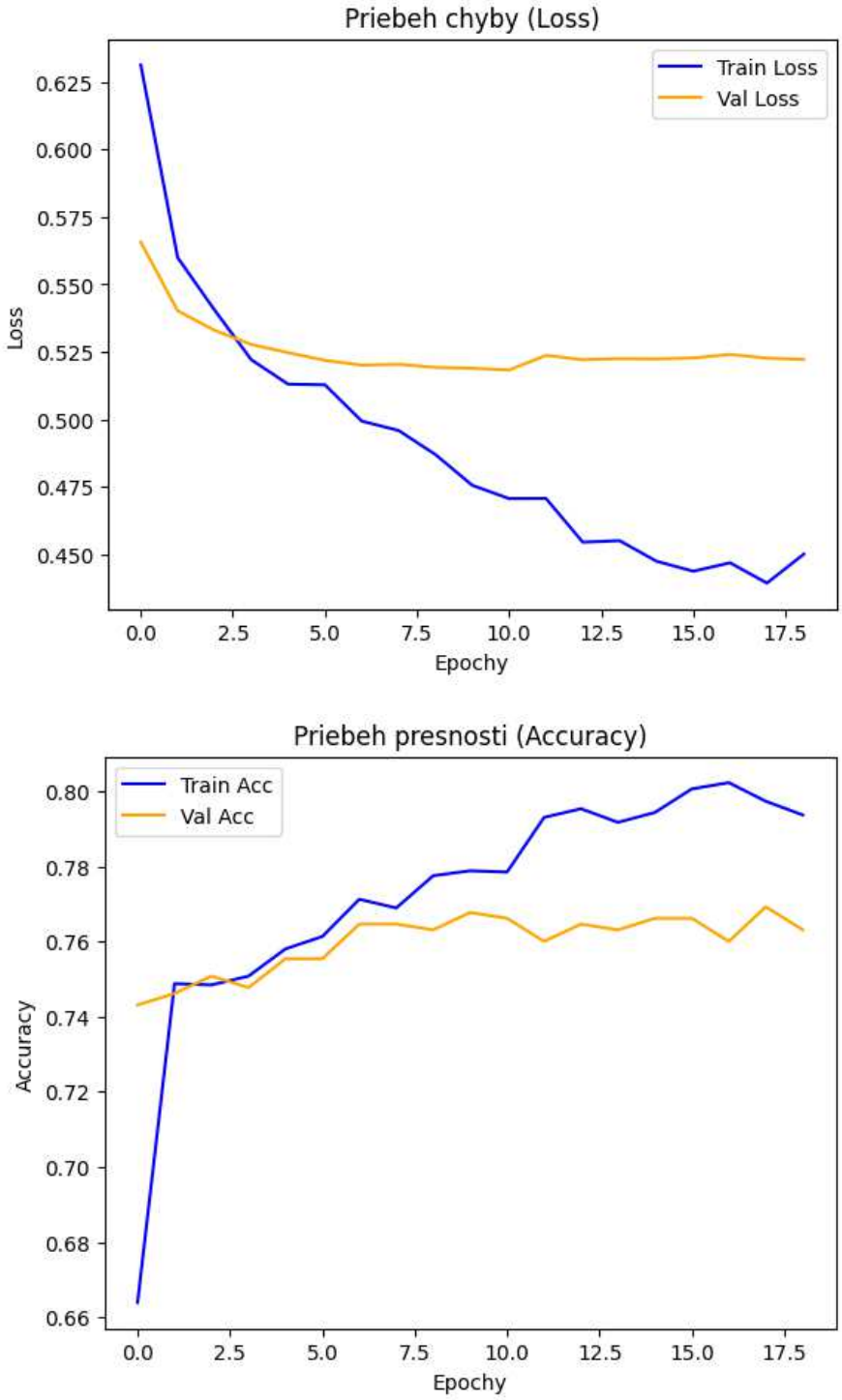
```
Accuracy (Presnosť):  0.7415
Precision:            0.7869
Recall (Senzitivita): 0.8944
F1 Score:             0.8372
```

Na pravej strane od zdrojového kódu je vidieť hodnoty metrík, ktoré nám vyšli pri teste. Presnosť modelu je 74,15%, čo vzhľadom na nižšiu kvalitu snímok s rozlíšením 10 metrov na pixelov je dobrý výsledok. Precíznosť modelu, ktorá znamená percento správne označených polí s výskytom škodcu, výsledok je z 78,69% správny. Senzitivita je najdôležitejšou metrikou pre moju oblasť, v ktorej dataset vyvíjam, pretože označuje počet oblastí odhalených z celkových napadnutých. Táto metrika minimalizuje riziko prehliadnutia nákazy a pri mojom testovaní dosiahla

[6] Nenormalizované hodnoty v intervale (-∞;∞), ktoré predstavujú mieru istoty

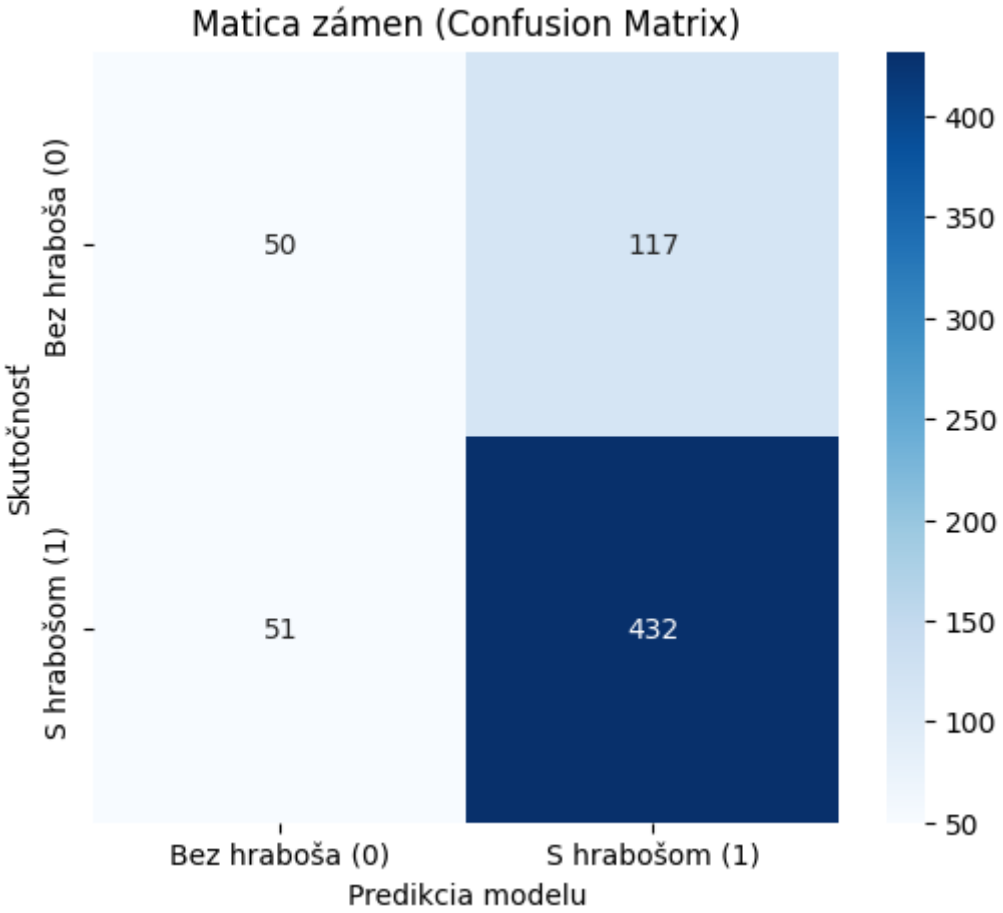
úrovne 89,44%, čo znamená odhalenie výskytu na pomere takmer 90 polí, pokiaľ by celkových bolo 100. F1-skóre ukazuje výkon algoritmu na základe senzitivite a precíznosti, a dosahuje 83,72%.

Pre grafické vyobrazenie správania modelu som vygeneroval grafy priebehu učenia pre chyby a presnosti spolu s maticou zámen.



Obrázok 8: Priebehy chýb a presností zobrazených grafmi

Z grafov je možné si povšimnúť, že sa model učil do 11. epochy, od ktorej validačná chyba začala jemne rásť. Priebeh chyby ukazuje stabilné klesanie a validačná chyba neklesá výrazne pod trénovaciu, čo vyznačuje tréning bez veľkého preúčania. Priebeh presnosti ukazuje rýchly nárast v začiatočných epochách, no okolo hodnoty 77% sa začína stávať stabilným, od ktorej je vidieť pri validačnej presnosti výkyvy, ktoré sú pomocou learning rate minimalizované.



Obrázok 9: Matica zámen na základe vyhodnotenia na testovacej množine

Matica zámen vykresľuje počty predikcií voči skutočným hodnotám. Z celkového počtu 650 záznamov tréningovej množiny datasetu model dokázal správne predikovať 50 prípadov bez výskytu a 432 prípadov s výskytom.

Chýb sa dopustil 51 pri predikcii poľa bez hraboša, aj keď v skutočnosti pole bolo obývané hrabošom. Táto hodnota môže byť spôsobená kvôli záznamom s príliš nízkou intenzitou výskytu hraboša, napríklad 10 nôr na hektár, pri ktorých si model nemusí byť istý, keďže to je veľmi nízke číslo. Na druhú stranu model označil 117 falošných poplachov, kedy predikoval pole s výskytom, aj keď v skutočnosti bolo bez výskytu. To znamená, že model radšej označí zdravé pole ako rizikové namiesto prehliadnutia výskytu, čo by značilo problém pre poľnohospodára. Tieto chybné predikcie znamenajú priestor pre ďalšie zlepšovanie.

Na konci vyhodnotenia som model uložil do súboru *model.pth*, ktorý obsahuje všetky váhy a biasy.

5 Diskusia

Táto práca mala za účel vytvoriť a overiť na základných AI modeloch multimodálny dataset. Špecificky sa táto problematika zabýva detekciou škodcu hraboša poľného na poliach s využitím terénnych hlásení o škodcovi, satelitnými dátami a meteorologickými údajmi. Model dosiahol na testovacej množine presnosť 74,15% a senzitivitu 89,14%, čo sa v oblasti agronómie môže brať ako vysoká kľúčová hodnota. Pri klasifikácii výskytu alebo predikcii poškodenia na poliach je dôležitá minimalizácia prehliadnutia skutočnej hrozby. Výsledky ukazujú, že sa z väčšiny prípadov je možné spoľahnúť na model, hoci aj za vyššieho množstva falošných poplachov, čo vyznačuje 78,69% precíznosti. Pri diaľkovom prieskume biologických škodcov je náročné rozlíšiť spektrálne zmeny spôsobené škodcami od rozlíšenia iných stresových faktorov, čím je napríklad sucho alebo zdravie.

5.1 Identifikované problémy

Najvýznamnejším limitom pri spracovaní tejto práce bol deficit satelitných snímok s vyšším rozlíšením. Satelit Sentinel-2 poskytuje najkvalitnejšie voľne dostupné dáta v súčasnej dobe, ktoré majú rozlíšenie 10 metrov na pixel, pre detekciu hraboša poľného je toto rozlíšenie problémové. Pokiaľ je nízke množstvo nôr na danom poli, je to takmer neviditeľné v takýchto snímkach, a preto sa musí model spoľahnúť viac na vegetačné indexy, ktoré nie sú špecifické iba pre hraboša. Pre presnú detekciu na úrovni jednotlivých rastlín alebo menších skupín je dôležité využiť snímkovanie až s centimetrovou presnosťou, ktoré satelity Sentinel-2 neposkytujú. S nízkou kvalitou snímok sa objavil problém s preučením pri tréningu modelu. Tento problém sa riešil pomocou optimalizácií, ktoré znížili overfitting, ale nedalo sa mu predísť úplne.

Ďalším problémom pri tvorbe datasetu bol nedostatok dát ohľadom plesne zemiakovej, ktorú na základe nízkeho množstva informácií bolo potrebné odstrániť. Nízky počet dát vznikol kvôli filtrácii datasetu kvôli vysokej oblačnosti alebo nízkej kvalite satelitných snímok. Týmto sa razantne znížil počet záznamov a bolo kvôli šumu, ako ktorý by pleseň zemiakovú model vyhodnotil, dáta s ňou spojené vymazať.

5.2 Možnosti zlepšenia

Pre zvýšenie presnosti modelu by bola vhodná integrácia snímok s vyšším rozlíšením. Najideálnejšou voľbou je využitie dronov vybavených multispektrálnymi kamerami. Drony sú schopné zbierať snímky s rozlíšením až v centimetroch, čo umožňuje detekciu konkrétnych nôr hraboša. Kombinácia UAV snímkovania s hlbokým učením prekonáva možnosti monitorovania pomocou satelitu pri detekcii lokálnych poškodení. Táto metóda je drahšia a nepokrýva tak veľké územia ako satelit, ale môže slúžiť ako kontrola alebo dolaďovanie modelov označených modelom.

6 Záver

Cieľom tejto bakalárskej práce bolo navrhnúť multimodálny dataset a overenie jeho kvality pre detekciu škodcov v oblasti agronómie. Práca pozostávala z integrácie troch dátových zdrojov. Prvým zdrojom boli terénne hlásenia z ústavu ÚKZÚZ, druhým boli satelitné snímky a vegetačné indexy zo satelitu Sentinel-2 cez Copernicus Data Space Ecosystem za pomoci Sentinel Hub, a tretím zdrojom boli meteorologické údaje pomocou Open-Meteo API. Dáta museli prejsť komplexnými úpravami, filtráciou záznamov s chýbajúcimi dátami, filtrovaním nevyhovujúcich snímok, anotáciou, vyriešením nevyváženosti tried, kontrolou párov snímok a záznamov, normálizáciou a rozdelením na podmnožiny.

Podarilo sa vytvoriť robustný dataset s výskytom hraboša poľného na území Českej republiky v rokoch 2023 a 2024. Dataset pozostával z obrazovej časti, ktorú reprezentovali satelitné snímky, a tabuľkovej časti, ktorá obsahovala ostatné dáta. Tieto dáta boli trénované konvolučnou neurónovou sieťou a základným modelom hlbokého učenia.

Vytvorený dataset predstavuje dobrý základ pre ďalší výzkum v oblasti poľnohospodárstva a môže byť využitý pre vývoj pokročilejších predikčných nástrojov, ktoré budú optimalizovať zdravie rastlín a znižovať škody v ekonómii spôsobené škodcami.

7 Zoznamy a literatúra

Zoznam obrázkov

Obrázok 1: Porovnanie neurónu a umelého neurónu. Upravené podľa (Haykin, 2009) 12
Obrázok 2: Aktivačné funkcie Sigmoid a ReLU. Prevzaté z portálu Medium (2024) 13
Obrázok 3: Vrstvy neurónovej siete. Upravené z portálu GeeksForGeeks (2025). 14
Obrázok 4: Architektúra konvolučnej neurónovej siete. Upravené z portálu Analytics Vidhya (2025) 16
Obrázok 5: Vzorové GEE snímky na základe image_path v datasete. 31
Obrázok 6: Vzorové rušivé GEE snímky 31
Obrázok 7: Vzorová snímka z nového zdroja CDSE 39
Obrázok 8: Priebehy chýb a presností zobrazených grafmi 54
Obrázok 9: Matica zámen na základe vyhodnotenia na testovacej množine 55

Zoznam tabuliek

Tabuľka 1: Prehľad vegetačných indexov. Spracované podľa Giovos et al. (2021), Muruganantham (2022) a India Remote Sensing (2025). 10
Tabuľka 2: Matica zámen. Upravené podľa Sathyanarayanan & Tantri (2024) 18
Tabuľka 3: Vzorové záznamy zo súboru export_vyskyty2024.csv. 26
Tabuľka 4: Vzorové záznamy zo súboru satellite_locations.json. 28
Tabuľka 5: Vzorové záznamy súboru results_complete.csv 30
Tabuľka 6: Zoznam parametrov a ich popis podľa Open-Meteo API, Haverkort (2011) a Valerio et al. (2020). 33
Tabuľka 7: Vzorové záznamy súboru results_enriched.csv spojené s meteorologickými údajmi. 35
Tabuľka 8: Vzorový záznam z tabuľkovej časti datasetu prepojeného so snímkou na základe image_path. 40
Tabuľka 9: Finálne parametre textovej časti datasetu. 44

Literatúra

Earth Engine documentation: https://developers.google.com/earth-engine/guides.

Pillow documentation: https://pillow.readthedocs.io/en/stable/.

Scikit-Learn documentation: https://scikit-learn.org/stable/supervised_learning.html.

PyTorch documentation: https://docs.pytorch.org/docs/stable/index.html.

GeeksForGeeks Neural Network Architectue image: https://www.geeksforgeeks.org/artificial-intelligence/artificial-neural-networks-and-its-applications/

Skendžić Sandra, Zovko Monika, Živković P. Ivana, Lešić Vinko & Lemić Darija (2021). „The Impact of Climate Change on Agricultural Insect Pests". In: *Insects 2021*. DOI: 10.3390/insects12050440.

Allu R. Ayyappa & Mesapam Shashi (2024). „Impact of remote sensing data fusion on agriculture application: A review". In: *European Journal of Agronomy.* DOI: 10.1016/j.eja.2024.127478.

Oerke E. C. (2006). „Crop losses to pests". In: *The Journal of Agricultural Science*. DOI: 10.1017/S0021859605005708.

Mulla J. David (2012). „Twenty five years of remote sensing in precision agriculture: Key advances and remaining knowledge gaps". In: *Biosystems Engineering*. ISSN: 1537-5110, DOI: 10.1016/j.biosystemseng.2012.08.009.

Xu Sheng-He & Wang Sai (2025). „An intelligent identification for pest and disease detection in wheat leaf based on environmental data using multimodal data fusion". In: *Frontiers in Plant Science.* DOI: 10.3389/fpls.2025.1608515.

Valerio Francesco, Ferreira Eduardo, Godinho Sérgio, Pita Ricardo, Mira António, Fernandes Nelson & Santos M. Sara (2020). „Predicting Microhabitat Suitability for an Endangered Small Mammal Using Sentinel-2 Data". In: *Remote Sensing,* DOI: 10.3390/rs12030562.

Ústřední kontrolní a zkušební ústav zemědělský. *Rostlinolékařský portál* [online]. [cit. 2025-04-24]. URL: https://ukzuz.gov.cz/public/portal/ukzuz.

Haverkort A. J. (1990). „Ecology of Potato Cropping Systems in Relation to Latitude and Altitude". In: *Agricultural Systems 32*, ISSN: 0308-521X.

Fernández I. Claudio, Leblon Brigitte, Haddadi Ata, Wang Keri & Wang Jinfei (2020). „Potato Late Blight Detection at the Leaf and Canopy Levels in the Red and Red-Edge Spectral Regions". In: *Remote Sensing,* DOI: 10.3390/rs12081292.

Lillesand M. Thomas, Kiefer W. Ralph & Chipman W. Jonathan (2015). „Remote Sensing and image interpretation". *7. vyd. Hoboken: John Wiley & Sons.* ISBN: 978-1-118-34328-9.

Drusch Matthias, Bello D. Umberto, Carlier Sébastien, Colin Olivier, Fernandez Veronica, Gascon Ferran, Hoersch Bainca, Isola Claudio, Laberinti Paolo, Martimort Philippe, Meygret Arnaud, Spoto François, Sy Omar, Marchese Filippo & Bargellini Pierluigi (2012). „Sentinel-2: ESA's Optical High-Resolution Mission for GMES Operational Services". In: *remote Sensing in Environment*, ISSN: 0034-4257, DOI: 10.1016/j.rse.2011.11.026

Giovos Rigas, Tassopoulos Dimitrios, Kalivas Dionissios, Lougkos Nestor & Priovolou Anastasia (2021). „Remote Sensing Vegetation Indices in Viticulture: A Critical Review“. In: *Agriculture*, DOI: 10.3390/agriculture11050457

„Complete Guide to GIS Spectral Indices: NDVI, NDWI, NDBI & More“ (2025). *India Remote Sensing* [online]. [cit. 2025-06-17]. URL: https://www.indiaremotesensing.com/2025/08/vegetation-and-other-indices-formulas.html

Muruganantham Priyanga, Wibowo Santoso, Grandhi Srimannarayana, Samrat H. Nahidul & Islam Nahina (2022). „A systematic Literature Review on Crop Yield Prediction with Deep Learning and Remote Sensing“. In: *Remote Sensing,* DOI: 10.3390/rs14091990

Piuri Vincenzo, Raj Sandeep, Genovese Angelo & Srivastava Rajshree (2020). „Trends in deep learning methodologies: algorithms, applications, and systems“. *Londýn: Elsevier.* ISBN: 978-0-12-822226-3.

Mitchell Tom (1997). „Machine learning“. *New York: McGraw-Hill.* ISBN: 0-07-042807-7.

Goodfellow Ian, Bengio Yoshua & Aaron Courville (2016). „Deep Learning“. *Cambridge, MA: MIT Press*. ISBN: 978-0-262-03561-3.

Han Su-Hyun, Ko W. Kim, SangYun Kim & Young C. Youn (2018). „Artificial Neural Network: Understanding the Basic Concepts without Mathematics“. In: *Dementia and Neurocognitive Disorders.* DOI: 10.12779/dnd.2018.17.3.83.

Dubey R. Shiv, Singh K. Satish, Chaudhuri & B. Bidyut (2022). „Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark“. In: *Neurocomputing.* arXiv: 2109.14545v3.

Medium Activation Functions image: https://medium.com/@srivastavashivansh8922/understanding-the-difference-between-relu-and-sigmoid-activation-functions-in-deep-learning-33b280fc2071

Haykin Simon (2009). „Neural Networks and Learning Machines“. *3. vyd. Pearson Education*, ISBN: 978-0-13-147139-9.

Boulila Wadii, Alshanqiti Eman, Alzahem Ayyub, Koubaa Anis & Mlaiki Nabil (2024). „An Effective Weight Initialization Method For Deep Learning: Application to Satellite Image Classification“. arXiv: 2406.00348v1.

Aggarwal C. Charu (2021). „Artificial Intelligence: A Textbook“. *Cham: Springer*, ISBN: 978-3-030-72356-9.

Elharrous Omar, Bechqito Yassine, Badidi Elarbi, Mahmood Yasir, Serhani A. Mohamed & Riffi Jamal (2025). „Loss Functions in Deep Learning: A Comprehensive Review“. arXiv: 2504.04242v1.

Noor M. H. Mohd & Ige O. Ayokunle (2025). „A Survey on State-of-the-art Deep Learning Applications and Challenges“. In: *Engineering Applications of Artificial Intelligence,* arXiv: 2403.17561v9.

Zhang Aston, Lipton C. Zachary, Li Mu & Smola J. Alexander (2023). „Dive into Deep Learning“. *Cambridge University Press*, arXiv: 2106.11342v5.

Krizhevsky Alex, Sutskever Ilya & Hinton E. Geoffrey (2012). „ImageNet Classification with Deep Convolutional Neural Networks“. In: *Advances in Neural Information Processing Systems,* DOI: 10.1145/3065386

Lecun Yann, Bengio Yoshua & Hinton Geoffrey (2015). „Deep learning“. In: *Nature*, DOI:10.1038/nature14539.

Kamilaris Andreas & Prenafeta-Boldú X. Francesc (2018). „Deep learning in agriculture: A survey“. In: *Computers and Electronics in Agriculture*, DOI: 10.1016/j.compag.2018.02.016.

Analytics Vidhya Convolutional Neural Network image: https://www.analyticsvidhya.com/blog/2020/10/what-is-the-convolutional-neural-network-architecture/

Vaswani Ashish, Shazeer Noam, Jones Llion, Parmar Niki, Uszkoreit Jakob, Gomez N. Aidan, Kaiser Łukasz & Polosukhin Illia (2017). „Attention is All You Need“. arXiv: 1706.03762v7

Zuo Shuangquan, Xiao Yun, Chang Xiaojun & Wang Xuanhong (2022). „Vision transformers for dense prediction: A survey“. In: *Knowledge-Based Systems*, DOI: 10.1016/j.knosys.2022.109552.

Yosinski Jason, Clune Jeff, Bengio Yoshua & Lipson Hod (2014). „How transferable are features in deep neural networks?“. In: *Advances in Neural Information Processing Systems*, arXiv: 1411.1792

Gerón Aurélien (2019). „Hands-on Machine Learning with Sci-kit Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems“. *2. vyd. Sebastopol: O'Reilly Media,* ISBN: 978-1-492-03264-9.

Srivastava Nitish, Hinton Geoffrey, Krizhevsky Alex, Sutskever Ilya & Salakhutdinov Ruslan (2014). „Dropout: A Simple Way to Prevent Neural Networks from Overfitting“. In: *Journal of Machine Learning Research*, URL: https://www.jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf?utm_content=buffer79b4.

Prechelt Lutz (1998). „Early Stopping – but when?“. In: *Neural Networks: Tricks of the Trade,* URL: https://page.mi.fu-berlin.de/prechelt/Biblio/stop_tricks1997.pdf.

Sathyanarayanan Swaminathan & Tantri B. Roopashri (2024). „Confusion Matrix-Based Performance Evaluation Metrics“. In: *African Journal of Biomedical Research,* DOI: 10.53555/AJBR.v27i4S.4345.

Vakili Meysam, Ghamsari Mohammad & Rezaei Masoumeh (2020). „Performance Analysis and Comparison of Machine and Deep Learning Algorithms for IoT Data Classification“. arXiv: 2001.09636v1.

Yang Wenxuan, Zhang Hanyu, Tan Weimin, Sun Yuqi & Yan Bo (2024). „A Self-Supervised Paradigm for Data-Efficient Medical Foundation Model Pre-training: V-information Optimization Framework“. arXiv: 2408.07107.

Russel J. Stuart & Norvig Peter (2021). „Artificial Intelligence: Modern Approach“. *4. vyd. Pearson,* ISBN: 978-0-13-461099-3.

Alamu Raphael (2023). „Automating Data Annotation and Labeling with AI: A Machine Learning Perspective“. URL: https://www.researchgate.net/publication/389628485_Automating_Data_Annotation_and_Labeling_with_AI_A_Machine_Learning_Perspective.

Kotsiantis B. Sotiris, Kanellopoulos Dimitris & Pintelas P. E. (2006). „Data Preprocessing for Supervised Learning“. In: *International Journal of Computer Science*, ISSN: 1306-4428.

Han Jiawei, Kamber Micheline & Pei Jian (2011). „Data Mining: Concepts and Techniques“. *3. vyd. Waltham: Elsevier,* ISBN: 978-0-12-381479-1.

Little J. A. Roderick & Rubin B. Donald (2020). „Statistical Analysis with Missing Data“. *3. vyd. Hoboken: John Wiley & Sons,* ISBN: 978-1-11-948226-0.

Hastie Trevor, Tibshirani Robert & Friedman Jerome (2009). „The Elements of Statistical Learning: Data Mining, Inference, and Prediction“. *2. vyd. New York: Springer,* ISBN: 978-0-387-84858-7.

Fernández Alberto, García Salvador, Galar Mikel, Prati C. Ronaldo, Krawczyk Bartosz & Herrera Francesco (2018). „Learning from Imbalanced Data Sets“. Cham, *Switzerland: Springer*, ISBN: 978-3-319-98074-4.

He Haibo & Garcia A. Edwardo (2009). „Learning from Imbalanced Data“. In: *IEEE Transactions on Knowledge and Data Engineering*, DOI: 10.1109/TKDE.2008.239

Kumar Varun, Choudhary Ashutosh & Cho Eunah (2020). „Data Augmentation Using Pre-Trained Transformer Models“. arXiv: 2003.02245.

He Kaiming, Zhang Xiangyu, Ren Shaoqing & Sun Jian (2015). „Deep Residual Learning for Image Recognition“. arXiv: 1512.03385v1.

Popescu Marius-Constantin, Perescu-Popescu Liliana, Balas E. Valentina & Mastorakis Nikos (2009). „Multilayer Perceptron and Neural Networks“. ISSN: 1109-2734.

Prílohy

Dataset a zdrojové kódy sú priložené v súbor **xkosar_prilohy.zip**. Tento súbor sa skladá z priečinkov podľa postupu spracovania očíslovaných od najstaršieho po najnovších, v kterých sú zdrojové kódy. Vedľa týchto priečinkov je uložený dataset s ďalším priečinkom obsahujúcim satelitné snímky.

SEZNAMY

Vložit potřebné údaje